UNIP

UNIVERSIDADE PAULISTA

Programação Orientada a Objetos I

Autor: Prof. Tarcísio de Souza Peres
Colaboradores: Prof. Angel Antonio Gonzalez Martinez
Profa. Larissa Rodrigues Damiani

Professor conteudista: Tarcísio de Souza Peres

Tarcísio de Souza Peres é formado em Engenharia da Computação e mestre pela Unicamp. Atuou como pesquisador no Human Cancer Genome Project, onde utilizou algoritmos e programação orientada a objetos. Tem MBA em Gestão de TI pela Fiap, formação em Gestão de Projetos pela FIA/USP e cursou Gestão Estratégica e Competitividade pela Fundação Dom Cabral. Com perfil multidisciplinar, tem experiência nas áreas de tecnologia, saúde, governança, novos negócios e inovação, tendo trabalhado em multinacionais e órgãos públicos. É certificado pelo PMI, Cobit e escreveu livro sobre mercado financeiro. Além de ser empreendedor (startups de inovação em tecnologia), professor da UNIP, FGV e do Centro Paula Souza.

Dados Internacionais de Catalogação na Publicação (CIP)

P437p Peres, Tarcísio de Souza.

Programação Orientada a Objetos I / Tarcísio de Souza Peres. – São Paulo: Editora Sol, 2024.

268 p., il.

Nota: este volume está publicado nos Cadernos de Estudos e Pesquisas da UNIP, Série Didática, ISSN 1517-9230.

1. Programação. 2. Linguagem. 3. Algoritmos. I. Título.

CDU 681.3.062

U519.68 – 24

© Todos os direitos reservados. Nenhuma parte desta obra pode ser reproduzida ou transmitida por qualquer forma e/ou quaisquer meios (eletrônico, incluindo fotocópia e gravação) ou arquivada em qualquer sistema ou banco de dados sem permissão escrita da Universidade Paulista.

Profa. Sandra Miessa
Reitora

Profa. Dra. Marilia Ancona Lopez
Vice-Reitora de Graduação

Profa. Dra. Marina Ancona Lopez Soligo
Vice-Reitora de Pós-Graduação e Pesquisa

Profa. Dra. Claudia Meucci Andreatini
Vice-Reitora de Administração e Finanças

Prof. Dr. Paschoal Laercio Armonia
Vice-Reitor de Extensão

Prof. Fábio Romeu de Carvalho
Vice-Reitor de Planejamento

Profa. Melânia Dalla Torre
Vice-Reitora das Unidades Universitárias

Profa. Silvia Gomes Miessa
Vice-Reitora de Recursos Humanos e de Pessoal

Profa. Laura Ancona Lee
Vice-Reitora de Relações Internacionais

Prof. Marcus Vinícius Mathias
Vice-Reitor de Assuntos da Comunidade Universitária

UNIP EaD

Profa. Elisabete Brihy
Profa. M. Isabel Cristina Satie Yoshida Tonetto
Prof. M. Ivan Daliberto Frugoli
Prof. Dr. Luiz Felipe Scabar

Material Didático

Comissão editorial:
Profa. Dra. Christiane Mazur Doi
Profa. Dra. Ronilda Ribeiro

Apoio:
Profa. Cláudia Regina Baptista
Profa. M. Deise Alcantara Carreiro
Profa. Ana Paula Tôrres de Novaes Menezes

Projeto gráfico:
Prof. Alexandre Ponzetto

Revisão:
Caio Ramalho
Kleber Souza

Sumário

Programação Orientada a Objetos I

APRESENTAÇÃO 9
INTRODUÇÃO 10

Unidade I

1 BASES DA PROGRAMAÇÃO ORIENTADA A OBJETOS 13
1.1 Fundamentação e motivação 13
1.2 Classe e objeto 16
1.3 Os quatro pilares: abstração, encapsulamento, herança e polimorfismo 17
1.3.1 Detalhes de cada pilar 18
1.4 Breve histórico 29
2 ORIENTAÇÃO A OBJETOS EM C# 32
2.1 Criando classes em C# 32
2.1.1 Private 36
2.1.2 Public 37
2.1.3 Protected 37
2.1.4 Internal 38
2.1.5 Protected Internal 38
2.1.6 Private protected (introduzido no C# 7.2) 38
2.2 Atributos, campos e constantes 39
2.3 Métodos 41
2.4 Construtores e desconstrutores 53
2.5 Criação de objetos 57
2.6 Referência this 59
2.7 Classes estáticas 60
2.8 Interfaces e classes abstratas 62
2.9 Hierarquia de classes 65
2.10 Herança (simples e múltipla) e polimorfismo 66

Unidade II

3 DETALHAMENTO DA LINGUAGEM C# 80
3.1 Sintaxe 81
3.2 Namespaces 86
3.3 Tipos numéricos 88
3.3.1 System.Math 90
3.3.2 System.Numerics.BigInteger 91
3.3.3 System.Decimal 91

3.4 Tipos booleanos 92
3.5 Cadeias de caracteres 95
3.6 Variáveis 100
3.7 Operadores e expressões 102
3.8 Declarações 105
3.9 Vetores 110
3.10 Enumeração 115
3.11 Estruturas 118
3.12 Tipo objeto 121
3.13 Entrada e saída 126
4 RELAÇÃO ENTRE C# E .NET 137
4.1 CLR e BCL 139
4.2 Camadas de aplicação 141
4.3 Trabalhando com números 152
4.3.1 Convert 156
4.3.2 Numerics 161
4.3.3 Math 166
4.4 Trabalhando com texto 166
4.5 Trabalhando com datas 171
4.6 Conversões 174
4.7 Breve histórico da linguagem C# e do .NET 177

Unidade III

5 APROFUNDAMENTO NA LINGUAGEM C# 186
5.1 Exceções 187
5.2 Delegação 189
5.3 Tuplas 191
5.4 Coleções 193
5.5 Expressões regulares 194
6 MELHORANDO A PROGRAMAÇÃO 197
6.1 Boas práticas de codificação 198
6.2 Princípios SOLID, DRY, KISS, YAGNI e Occam 201
6.3 Design patterns 204

Unidade IV

7 TÓPICOS ESPECIAIS EM C# 217
7.1 Expressões lambda 218
7.2 LINQ: consultas e operadores 220
7.3 Disposal e Garbage Collection 224
7.4 Debugging e tracing 225
7.5 Patterns 227
7.6 Threading 230

7.7 Tasks....236
7.8 Funções e padrões assíncronos....240
8 CLASSES E ALGORITMOS....241
8.1 Ordenação de vetores e listas – seleção, inserção, bubble sort, quick sort....242
8.2 Pilhas e filas – Torre de Hanói....245
8.3 Dicionários e conjuntos – tabelas de hash....251
8.4 Árvores – binária, busca binária, autobalanceada....254
8.5 Heaps – binário, binomial, Fibonacci....258
8.6 Grafos – Kruskal, Prim e Dijkstra....259

APRESENTAÇÃO

O objetivo geral da disciplina *Programação Orientada a Objetos I* é introduzir conceitos básicos da orientação a objeto e capacitar o aluno a implementar programas do tipo. Já os objetivos específicos são apresentar e consolidar os princípios desse modelo de programação e capacitar o aluno a implementar aplicações utilizando linguagem C#.

Durante anos a atividade do desenvolvedor de software esteve atrelada a um paradigma conhecido como **estruturado**, e a atividade principal do profissional desenvolvedor de sistemas era a programação em si. Nesse contexto, linguagens de programação como Cobol, Clipper, Pascal e C dominaram o mercado.

Esse paradigma estruturado é simples de aprender e aplicar – motivo pelo qual, nos cursos de programação de diversas universidades nacionais e internacionais, os alunos aprendem primeiramente linguagens estruturadas, em especial a linguagem de programação C, comumente adotada por ser uma referência robusta e completa em sua proposta.

Com o passar dos anos, as linguagens de programação evoluíram, e foi criado o paradigma orientado a objetos, que, além de ter as vantagens da programação estruturada, abria novos horizontes de programação. Consequentemente, surgiu uma nova família de linguagens usufruindo dessa nova perspectiva: Smalltalk, Java, C++ e, recentemente, C# são exemplares dessa nova família de linguagens orientadas a objetos.

Diferente da programação estruturada – que contém um paradigma mais simples de aprender –, a programação orientada a objetos (POO) demanda uma curva de aprendizado mais longa: é necessário entender alguns conceitos-chave antes de iniciar as atividades da programação propriamente dita. O aprendiz precisa de mais embasamento teórico antes da prática.

Portanto, nesta disciplina o aluno conhecerá os quatros pilares desse novo paradigma – abstração, encapsulamento, herança e polimorfismo – e os blocos básicos de construção de um sistema orientado a objetos: classes e objetos. Tais conceitos são fundamentais para o atual processo de desenvolvimento dos aplicativos tanto para celular quanto para dispositivos inteligentes (na chamada internet das coisas) e inteligência artificial, entre outras aplicações modernas. Não é possível atuar profissionalmente com tais tecnologias sem antes entender os conceitos aqui apresentados; assim, absorvido esse conhecimento, o aluno estará apto a aplicá-las na prática com a linguagem de programação apropriada.

Esta disciplina apresenta especificamente a linguagem C#, começando diretamente pela aplicação dos conceitos-chave em POO (classes, objetos, métodos, herança etc.) e, posteriormente, revisando conceitos de programação estruturada (tipos numéricos, booleanos, operadores, condicionais etc.). A linguagem C#, além de ser mais moderna se comparada a outras linguagens, tem ampla utilização empresarial, dadas suas características de interoperabilidade, segurança e robustez, fundamentais para o mundo real do desenvolvimento de sistemas e aplicações.

INTRODUÇÃO

De forma simplificada, podemos entender um telefone celular (smartphone) ou um computador como máquinas compostas de diversas partes físicas (hardware). Esses componentes eletrônicos têm características próprias e funcionalidades específicas: uma impressora deve saber lidar com movimentos mecânicos de puxar uma folha, armazenar caracteres a imprimir, jatear a tinta com certa velocidade para não borrar a folha, entre outras tarefas. Já um mouse deve saber lidar com coordenadas espaciais (x, y), calcular a velocidade de movimentação da mão do usuário por sensores luminosos, além de transmitir a informação de que um botão foi pressionado (clique). Similarmente, a tela sensível ao toque de um smartphone deve transmitir coordenadas, velocidade de movimentos e toques. Esses exemplos são de dispositivos visíveis no nosso cotidiano, mas devemos recordar também das tarefas de componentes internos, como um microprocessador (CPU), uma memória (RAM) ou um disco que armazena dados (SSD).

No contexto atual existem diversos outros equipamentos – chamados de equipamentos inteligentes –, com processadores, memória, espaço de armazenamento de dados e conexão com rede (internet). Seja uma geladeira, máquina de lavar ou drone, todos executam tarefas específicas e têm características próprias de trabalho. O universo de possibilidades de equipamentos conectados pela rede é chamado *internet of things* (IoT – em português, internet das coisas). Existe um verdadeiro ecossistema de dispositivos interconectados.

Todas essas tarefas, realizadas por diferentes elementos de hardware, podem ser controladas por instruções pré-programadas. Cada componente tem um conjunto diferente de instruções, definido por seu fabricante. Portanto, para controlar qualquer dispositivo, é preciso acessar e manipular cada um desses conjuntos de instruções.

Como um computador (ou smartphone) tem muitas partes (CPU, memória, monitor, teclado etc.), existe um sistema robusto que utiliza esse conjunto de instruções e controla automaticamente os componentes de hardware, facilitando sua utilização; esse sistema é chamado de **sistema operacional**. Alguns sistemas operacionais populares são Unix, Linux, MacOS, Microsoft Windows, iOS e Android, que permitem executar instruções mais complexas utilizando recursos computacionais, descritas pelas chamadas **linguagens de programação**. Assim, podemos dizer que programar é o ato de escrever um texto seguindo o padrão definido por alguma linguagem de programação inteligível para o sistema operacional ou pelo controlador do hardware, permitindo a execução de tarefas e uso dos recursos computacionais disponíveis. Tal texto – tecnicamente chamado **código-fonte** – dá origem ao programa ou software. Esse texto não é escrito em português, e sim na linguagem de programação escolhida.

Neste livro-texto será apresentado o C# (lê-se "C sharp"), uma das linguagens de programação mais relevantes da atualidade. Portanto, se você pretende criar um aplicativo de celular, um software que interage com sua geladeira inteligente ou um programa que ajude sua empresa em determinado negócio, é necessário entender e programar em C#. Como essa linguagem é orientada a objetos, descreveremos e explicaremos na unidade I os conceitos-chave desse paradigma, com o objetivo de extrair o máximo do potencial da linguagem C#. A mesma unidade contém exercícios práticos de programação para que o aluno consiga escrever seus primeiros programas.

A unidade II será iniciada com os conceitos da linguagem estruturada já familiares ao aluno e envolvem tipos de dados, operadores e controle do fluxo do programa. A seção 4 explorará em mais detalhes o .NET, plataforma da Microsoft que desenvolve diversas aplicações, incluindo as escritas em C#.

A partir de conceitos e práticas abordadas ao longo do livro-texto, a unidade III aprofundará certas propriedades da linguagem que garantem uma melhor qualidade da codificação e usam boas práticas de programação e processos mais modernos de desenvolvimento, como padrões de design e princípios de programação.

Por fim, a unidade IV será composta de tópicos especiais em programação C# e algoritmos específicos da computação. Esses algoritmos são amplamente utilizados nas empresas e servem como modelos de aplicação em diversas áreas do conhecimento. Para melhor fixação, diversos exercícios e exemplos práticos serão apresentados. Esses assuntos serão exibidos ao final do livro-texto pois exigem maior desenvoltura do aluno na linguagem e permitem o aprimoramento técnico do seu aprendizado na disciplina.

Unidade I

1 BASES DA PROGRAMAÇÃO ORIENTADA A OBJETOS

Esta seção fundamenta a POO e as motivações para seu surgimento, utilização crescente e ampla disseminação nas atividades de programação e desenvolvimento de sistemas da atualidade. Compreender o contexto e a relevância da criação e adoção do paradigma orientado a objetos estimula o leitor a assimilar seus conceitos-chave: classes, objetos, abstração, encapsulamento, herança, polimorfismo e sua utilização no mundo real. Ao final da seção, será relatado brevemente o contexto histórico da orientação a objetos.

1.1 Fundamentação e motivação

Como vimos, um programa é gerado a partir de um código-fonte, que é um conjunto de instruções escrito em determinada linguagem de programação. A primeira linguagem de programação surgiu em 1883 e foi criada pela primeira programadora do mundo, Ada Lovelace (Huskey; Huskey, 1980), mas somente em 1949 surgiu uma linguagem de uso comercial, denominada Assembly (Wilkes, 1949); nela os primeiros códigos-fonte eram estruturados como uma lista de instruções a serem executadas sequencialmente. Por exemplo, o código da figura 1 faz uma contagem de um até dez:

```
xor ecx, ecx
loop:
  inc ecx
  cmp ecx, 0xa
  jl loop
```

Figura 1 – Código-fonte escrito na linguagem Assembly para contar de um até dez

Nas décadas de 1950 e 1960 surgiram diversas outras linguagens (Autocode, Fortran, Algol, COBOL, LISP, BASIC e Pascal, entre outras). Essas linguagens seguiam o padrão de códigos-fonte escritos como uma de lista de instruções a serem executadas linearmente e de forma estruturada. Naquela época se popularizou o uso de linguagens de programação, e a atividade de codificação (produção de textos nas linguagens) floresceu de modo expressivo.

O aumento do número de programas e, consequentemente, de programadores resultou no crescimento proporcional de linhas de código e da sofisticação da lógica dos programas: os textos se tornavam cada vez maiores e mais complexos. Para ilustrar a questão da complexidade de um código, podemos compará-lo com a própria língua portuguesa. Uma lista de compras de supermercado de dez linhas é mais simples de entender que o soneto "Transforma-se o amador na cousa amada" (Camões, 1953), de 14 linhas; este por sua vez é mais fácil de entender que as 36 linhas do "Retrato", de Antonio Machado (1949), que por sua vez é mais simples que o conto "O nariz", de 28 páginas, de Nikolai Gogol (2008). Conforme aumentam o volume de linhas e a sofisticação do conteúdo do texto, a complexidade também aumenta.

A questão fica mais delicada se lembrarmos que, a partir de um certo tamanho, é necessário mais de uma pessoa trabalhando no mesmo texto. Tente visualizar, por exemplo, um texto sendo escrito simultaneamente por cinco pessoas. Surgem novos desafios relativos à padronização dos termos, sincronização das alterações, separação de tarefas, entre outras questões. Esses elementos presentes no "trabalho em equipe" também somam complexidade à própria complexidade já intrínseca do código-fonte.

Imagine alterar ou dar manutenção a um programa com 50 linhas de código. Parece uma tarefa tranquila, mesmo para um programador iniciante. Reflita agora sobre como dar manutenção a um código-fonte com mil linhas. A partir de sete mil linhas a tarefa já é penosa, mesmo para um programador experiente, já demandando a necessidade de mais de um programador no mesmo código-fonte. Muitos programas empresariais típicos romperam limites: 100 mil linhas de código, 500 mil, mais de um milhão, e progredindo.

Com o passar dos anos essa explosão de complexidade trouxe um problema: como continuar atendendo à demanda crescente (mais e mais empresas queriam novos programas e softwares) se o universo de codificação estava se tornando intricado, confuso, enrolado? A programação de computadores, à época, estava encontrando uma barreira de crescimento que foi fruto do seu próprio sucesso e disseminação em larga escala. Lembre-se que naquela época a computação era uma atividade onerosa, acessível basicamente às empresas e não ao cidadão comum. Isso significa que a prosperidade da atividade de programação se atrelava aos benefícios que a computação estava efetivamente trazendo a diversos setores empresariais.

Mais lucro para as empresas, mais orçamento, mais recursos computacionais, mais softwares, mais programadores, maior complexidade de código. O paradigma de programação estruturada não parecia se ajustar aos novos desafios. Após muitos anos surgiu também o termo **programação macarrônica** para se referir ao fenômeno de que alguns programas tinham um código-fonte "tão enrolado quanto um espaguete" (Steele, 1977).

Na segunda metade dos anos 1960, surgiu um novo paradigma com a proposta de lidar com a complexidade crescente dos programas de computador. Esse paradigma utiliza uma estratégia conhecida como "dividir para conquistar", ou seja, dividir um problema grande em problemas menores, e dividir novamente os problemas menores em subproblemas, e assim sucessivamente. Resolvido cada subproblema, o problema original estará resolvido.

Percebemos duas vantagens nessa abordagem:

- Não foi necessário resolver diretamente o problema original, que é complexo.
- Os subproblemas têm menor complexidade e podem ser distribuídos para diferentes programadores.

A estratégia "dividir para conquistar" não era nova naquela época: seu uso na computação já havia aparecido em um artigo em 1946 e ela já havia sido usada pelos babilônicos em 200 a.C. (Knuth, 1998). A novidade foi utilizá-la para lidar com a crescente complexidade nos códigos-fonte estruturados.

E assim surgiu a POO, que reduz o universo do problema em partes menores chamadas classes (ou objetos), e os programadores tornaram-se especialistas em modelar, criar e implementar tais objetos. Modelar implica que é necessário entender os requisitos de um software para definir quais serão os objetos e as relações (ou conversas) entre eles. O termo conversas refere-se ao fato de que tais objetos podem trocar mensagens entre si para executar ações e tarefas. Em outras palavras, existe um universo que deriva dos requisitos do cliente, onde são definidos (modelados) objetos que vão interagir por mensageria (no tópico 1.2 abordaremos as diferenças entre classes e objetos). Implementação é a programação em si e a disponibilização do software em funcionamento para que ele opere conforme as expectativas.

Em virtude disso, um iniciante em POO precisa, antes de programar, entender os conceitos que norteiam o paradigma orientado a objetos. Ele necessita entender como os objetos são modelados a partir dos requisitos e como implementá-los no código-fonte e na linguagem de programação adequada. Com essa mudança de conceitos, surgiram linguagens de programação específicas para essa finalidade; a pioneira em implementar objetos e classes foi a Simula 67. Outros exemplos são Smalltalk, Emerald, Java, C++ e C# (sendo C# o foco deste livro-texto).

Ao longo dos anos a POO se desenvolveu cada vez mais, e com isso o gerenciamento da complexidade foi adequadamente endereçado. Apenas para se ter uma ordem de grandeza, atualmente há jogos (games) com sessenta milhões de linhas de código, e a equipe responsável pelo seu desenvolvimento tem menos de 1.500 programadores, o que dá uma média de 40 mil linhas de código por programador. Em 2015 foi divulgado que os serviços de internet do Google ultrapassavam 2 bilhões de linhas de código, o que daria uma média impressionante de 100 mil linhas de código por programador nessa empresa.

Esses números ilustram a importância de o programador entender adequadamente as questões de modelagem e seu impacto na produtividade da tarefa de programação. Em 1995 foi criada a *unified modeling language* (UML – em português, linguagem de modelagem unificada), criada para apoiar a modelagem dos objetos antes da programação (Rumbaugh; Jacobson; Booch, 2004). A UML tem uma série de diagramas visuais para facilitar o entendimento dos requisitos e sua transformação em classes. Por se tratar de uma especificação unificada, seu uso é amplo e atualmente é o padrão de modelagem de empresas que atuam com POO.

Ainda há mercado de trabalho tanto para quem trabalha com linguagens estruturadas quanto orientadas a objetos. Atualmente diversos programas são escritos em programação estruturada. Isso varia conforme o setor e o país, mas em geral novos softwares têm sido escritos em POO, e alguns sistemas mais antigos são mantidos em programação estruturada, em especial porque substituir sistemas críticos e em pleno funcionamento nem sempre é justificável economicamente. No Brasil, por exemplo, o setor bancário ainda tem muitos softwares escritos na linguagem estruturada COBOL; já nas empresas nacionais de comércio eletrônico ou no setor de jogos digitais, a POO é predominante, e praticamente não há sistemas no paradigma estruturado. Assim, se consideramos empresas de diversos setores em escala global, constata-se avanço considerável no número de linhas de código escritas no paradigma orientado a objetos e um movimento de declínio acentuado do paradigma estruturado.

1.2 Classe e objeto

Como o próprio nome destaca, na POO o objeto é um conceito extremante relevante. Objeto é um ente, individualizado, do mundo real que pode ser utilizado na programação. Uma mesa, uma caneta, um boleto bancário, um cartão de crédito, um passe escolar podem ser considerados objetos na programação.

O conceito de objeto na programação é mais abrangente que no senso comum, quando associamos "coisas" a objetos e "seres" a outra categoria. No mundo real, não consideramos uma pessoa ou um gatinho como objetos, mas na POO eles podem ser. É perfeitamente possível ter uma pessoa ou gatinho modelado como objetos em um sistema computacional. Ainda é possível, sob a ótica computacional, que "pessoa" e "gato" sejam um único objeto, um exemplar (ou instância) de um "mamífero".

Discutiremos abstração no capítulo 1.3, mas é possível observar no mundo real que um conjunto de objetos pode ter similaridades, coisas em comum, como se fizessem parte de um coletivo maior ou agrupamento. O objeto caneta no bolso de uma pessoa é diferente do objeto caneta no estojo de outra pessoa, que é distinto do objeto caneta na vitrine de uma loja. Esses três objetos têm formato e tamanho diferente, marcas diferentes, donos diferentes, preço de aquisição diferente, quantidade de tinta ou carga diferente, vida útil distinta e talvez até cores diferentes. São entes individuais, portanto. Ainda assim, podemos chamar todos de caneta.

Em POO, podemos chamar caneta de CLASSE e cada indivíduo representante da classe (cada uma das canetas) de OBJETO ou instância da classe caneta. Recapitulando o exemplo anterior, temos uma classe (caneta) e três objetos do tipo caneta; então classe é um conjunto de características comuns a todos os indivíduos (instâncias ou objetos).

Além das **características** citadas (tamanho e formato, marca, proprietário, valor de compra, quantidade de carga, vida útil, cor da tinta), existem **ações** ou **comportamentos** comuns: é esperado que uma caneta escreva (ação escrever), e podemos transferir temporariamente a propriedade da caneta (ação emprestar), por exemplo. Em C#, assim como em outras linguagens de POO, os atributos e métodos são componentes fundamentais para a definição e o funcionamento de classes e objetos. Os **atributos** (também conhecidos como campos) são as **características** ou propriedades dos objetos de uma classe; eles representam os dados que um objeto armazena. Já os **métodos** definem comportamentos ou funcionalidades que os objetos de uma classe podem realizar.

O fato de existir uma classe chamada caneta também nos possibilita ter uma nova caneta se quisermos, e ela pode ser diferente das anteriores, uma vez que é um indivíduo "caneta" ou instância "caneta". Do ponto de vista da programação, a classe existe desde o momento em que o programador a digitou no código-fonte até o fim da vida do programa. Para ser mais preciso, a classe surge um pouco antes: durante a modelagem, antes da programação (mencionamos que as empresas usam UML para modelar as classes visualmente e posteriormente inseri-las no código-fonte).

Já o objeto tem um ciclo de vida diferente. Ele surge somente quando a programação do software foi finalizada e com o programa em execução. Dentro do código, o programador inseriu uma espécie de semente que vai brotar quando o programa é executado. De modo mais preciso: o objeto é criado na

memória do computador assim que o programa em **execução** cria uma nova instância. E o objeto tem vida efêmera: ele será removido da memória depois de cumprir seu papel. Lembre-se que no computador a memória é volátil; ela está disponível para os programas em execução e perdura enquanto o equipamento estiver ligado. Veremos também que, no caso do C# com .NET, há um mecanismo de remoção automática de objetos em memória. Caso o programador não queira perder as informações do objeto, deverá utilizar estratégias de persistência (salvar em arquivos ou em bancos de dados, por exemplo).

Retomando o exemplo da caneta: por se tratar de uma classe, ela continuará existindo enquanto o programa existir. Já a caneta preta, marca Acme, do João, que custou R$ 20,00, tem seis meses de vida e metade da carga, será destruída ao término da execução do programa caso não seja implementada a persistência no código-fonte. Outro ponto relevante: somente temos uma classe caneta, mas podemos criar diversas instâncias de canetas. O número máximo de objetos caneta que podemos criar dependerá do espaço em memória disponível para o programa e do tamanho em bits de cada caneta.

Em outras palavras, há três diferenças fundamentais entre classes e objetos:

1. O objeto é um representante da classe.

2. Podemos criar múltiplos objetos a partir de uma única classe.

3. Cada objeto tem vida efêmera na memória volátil.

Abordaremos mais detalhes da linguagem de programação C# no tópico 2.

1.3 Os quatro pilares: abstração, encapsulamento, herança e polimorfismo

Os quatro pilares da POO fornecem uma estrutura conceitual sólida e eficaz no desenvolvimento de software. Cada pilar aborda aspectos específicos dos desafios enfrentados ao projetar e criar sistemas de software complexos. A seguir, um resumo do propósito de cada um deles:

- **Abstração**: simplifica a representação de objetos do mundo real em software e permite que os desenvolvedores se concentrem nas características e comportamentos relevantes dos objetos, enquanto ignoram detalhes desnecessários. Isso simplifica o processo de modelagem e torna o código mais compreensível, gerenciável e flexível.

- **Encapsulamento**: protege e controla o acesso aos dados e funcionalidades internas de um objeto, evitando modificações não autorizadas e minimizando a dependência entre diferentes partes do código. Isso promove a segurança e a modularidade, permitindo que os detalhes internos de uma classe sejam alterados sem afetar o restante do sistema.

- **Herança**: promove a reutilização de código e cria hierarquias de classes que compartilham características e comportamentos comuns, além de evitar a duplicação de código ao permitir que uma classe derivada herde atributos e métodos de uma classe-base. A herança facilita a criação de estruturas mais complexas a partir de elementos mais simples, tornando o design mais eficiente e a manutenção mais fácil.

- **Polimorfismo**: aumenta a flexibilidade e a extensibilidade do código ao permitir que um método seja usado de diferentes maneiras dependendo do contexto. Isso é especialmente útil quando se lida com classes derivadas, onde diferentes implementações de um método podem ser aplicadas a diferentes tipos de objeto, simplificando a chamada de métodos e facilitando a adição de novos tipos de objeto sem afetar o restante do código.

No geral, os quatro pilares da POO trabalham em conjunto para fornecer um arcabouço de design que torna o desenvolvimento de software mais eficiente, modular e adaptável, ajudando desenvolvedores a criar sistemas que se assemelham à estrutura e à natureza dos objetos do mundo real, resultando em um software mais intuitivo e de qualidade. Os pilares não foram criados por uma única pessoa, mas desenvolvidos ao longo do tempo por vários pesquisadores e praticantes da ciência da computação. Eles foram consolidados como princípios fundamentais da POO devido à sua eficácia na resolução de desafios no desenvolvimento de software.

1.3.1 Detalhes de cada pilar

Abstração

Princípio fundamental da POO, representa objetos do mundo real de maneira simplificada, destacando apenas atributos e comportamentos relevantes para o contexto em que são modelados.

Lembrete

No tópico 1.2 abstraímos o objeto do mundo real "caneta" para exemplificar uma classe, chamada "Caneta".

O conceito de abstração tem raízes nas ciências da matemática e da filosofia. Historicamente, uma curiosa disputa filosófica se estendeu por aproximadamente dez séculos, do início da Idade Média até o final da Renascença, conhecida como Querela dos Universais, um debate filosófico sobre a natureza dos "universais" – termos ou conceitos que podem ser aplicados a múltiplos indivíduos. Os envolvidos questionavam se os universais eram entidades reais independentes (realismo) ou simplesmente construções mentais (nominalismo). Em termos mais simples, abordaram a pergunta: "Conceitos abstratos, como 'humanidade', existem no mundo real ou são apenas ideias em nossas mentes?".

Na POO, a abstração permite criar classes que armazenam informações essenciais e fornecem métodos para interagir com elas enquanto ocultam detalhes internos complexos, ajudando a organizar e gerenciar códigos de maneira mais eficiente e compreensível.

A seguir, um exemplo que abstrai uma caneta usando a linguagem C#. Começaremos definindo uma classe chamada Caneta, conforme a linha 1 da figura 2. Para definir essa classe usamos a palavra-chave **class** (veremos em detalhes essa palavra-chave no tópico 2.1).

```
1.  public class Caneta
2.  {
3.    private string cor;
4.    private bool tampada;
5.
6.    public Caneta(string cor)
7.    {
8.      this.cor = cor;
9.      this.tampada = true; // Inicialmente, a caneta está tampada
10.   }
11.
12.   public void Destampar()
13.   {
14.     if (tampada)
15.     {
16.         tampada = false;
17.         Console.WriteLine("A caneta foi destampada");
18.     }
19.     else
20.     {
21.       Console.WriteLine("A caneta já está destampada");
22.     }
23.   }
24.
25.   public void Escrever(string texto)
26.   {
27.     if (!tampada)
28.     {
29.       Console.WriteLine("Escrevendo: " + texto);
30.     }
31.     else
32.     {
33.       Console.WriteLine("Não é possível escrever, a caneta está
tampada.");
34.     }
35.   }
36.
37.   public void Tampar()
38.   {
39.     if (!tampada)
40.     {
41.         tampada = true;
42.         Console.WriteLine("A caneta foi tampada");
43.     }
44.     else
45.     {
46.       Console.WriteLine("A caneta já está tampada");
47.     }
48.   }
49. }
```

Figura 2 – Código-fonte escrito na linguagem C# que abstrai uma caneta

Por enquanto não se preocupe com os demais termos em inglês que aparecem no código-fonte nem com a sintaxe do C#, tampouco com a plataforma usada para escrevê-lo, se ele funciona adequadamente ou se precisa de complementos. Concentre-se nos detalhes da abstração. As linhas 3 e 4 da figura 2 definem que a caneta tem dois atributos: cor e tampada. Seria possível incluir outros atributos também (como preço ou data de fabricação).

Observação

Não existe uma abstração de caneta ideal, válida para todos os programas e aplicações; a classe Caneta é modelada a partir de um conjunto de requisitos. Seus atributos e métodos são extraídos do levantamento e análise dos requisitos de determinado projeto de software desenvolvido e podem variar de projeto para projeto, de cliente para cliente.

Tente pensar em outra classe Caneta: existem diversas possibilidades, e todas estarão corretas, desde que sejam compatíveis com o levantamento de requisitos e com as regras de negócio previamente capturadas. Se os requisitos buscam desenvolver um software de fábrica de canetas, provavelmente serão oportunos atributos como peso, material, dimensões e preço de custo. Todavia, se os requisitos buscam programar um software para loja de material de escritório ou papelaria, atributos como fabricante, preço de venda e margem podem ser mais importantes.

As linhas 12 a 23 definem um método chamado Destampar que, quando executado, destampa a caneta (caso a caneta esteja tampada) e exibe a mensagem "A caneta foi destampada.". Note que estar tampada, do ponto de vista da abstração da caneta, bem como de sua programação na linguagem C#, é ter no atributo tampada um valor verdadeiro (true). Similarmente, estar destampada é ter nesse mesmo atributo um valor falso (false). Finalmente, o método Destampar exibe outra mensagem, "A caneta já está destampada", se no atributo tampada estiver inserido o valor falso (afinal, não faz sentido destampar uma caneta sem tampa).

Perceba que até o momento abstraímos uma caneta com os atributos cor e tampada e com uma ação (método Destampar). O restante do código-fonte abstrai também outras duas ações ou comportamentos: Escrever e Tampar. Esse conjunto de atributos e métodos finaliza nossa abstração de caneta. Na linha 25 o método Escrever recebe uma informação (texto), a frase que alguém deseja escrever com a caneta. Para escrever, é necessário checar antes se ela está tampada. Por fim, o método Tampar coloca o valor verdadeiro (true) no atributo tampada, uma vez que a abstração da tampa é um valor lógico (verdadeiro/falso).

Lembrete

O trecho de código-fonte da figura 2, de 49 linhas, escrito em linguagem C#, abstrai um objeto do mundo real, captura algumas características (atributos) e ações ou comportamentos (métodos) desse objeto real. Além disso, o código-fonte define uma dinâmica própria (ou lógica) de como as ações se relacionam com os atributos e com elas se integram. Tudo isso dentro de uma classe chamada Caneta.

Cabe ressaltar que, até o presente momento, nenhuma caneta foi efetivamente criada no computador. Tenha em mente que o código-fonte é um texto estático: ele por si não faz nada. É necessário um processo de compilação do código-fonte que o transforme em arquivo binário. Esse arquivo, quando executado por algo (outro software) ou por alguém, gera instruções para o sistema operacional do computador, recrutando recursos computacionais como memória, processamento, conexão com redes e armazenamento.

Sendo assim, podemos dizer que, se esse código-fonte for compilado e executado, a caneta será criada? Atualmente, um computador pode controlar uma impressora 3D que imprime uma caneta de verdade, ou ainda controlar um robô em uma linha de produção de uma fábrica de canetas, mas isso não é possível somente com o código-fonte da figura 2. Nenhuma caneta real será criada porque o computador não gerará artefatos materiais com esse texto de 49 linhas.

Outrossim, nenhuma caneta virtual será criada, pois a abstração representa um tipo de receita, molde ou esquema. Lembre-se: a abstração cria o conceito de caneta para o computador, mas não cria nenhuma caneta efetivamente. A classe define o que é uma caneta e como ela deve funcionar, mas não gera um exemplar da "espécie" caneta, nem um "indivíduo" caneta.

Criaremos uma ou várias canetas adiante, quando detalharmos o conceito de objeto da POO. Por hora, podemos dizer que o objeto é a caneta criada na memória do computador. Cada caneta criada em memória é um objeto individual e único, e veremos que existe uma relação entre classe e objeto: o objeto é feito a partir da classe, possuindo seus atributos e métodos. Confundir classe com objeto seria o mesmo que confundir o lanche do cardápio com o lanche que o garçom trouxe na mesa após ter pedido no cardápio.

Encapsulamento

Refere-se à capacidade de ocultar os detalhes internos de um objeto e expor apenas uma interface pública. Em outras palavras, os atributos (dados) de um objeto são geralmente definidos como privados, e o acesso a esses atributos é controlado por métodos públicos. Isso ajuda a manter a integridade dos

dados e facilita a manutenção do código, pois as mudanças internas podem ser feitas sem afetar o código que utiliza o objeto.

Esse princípio permite ao programador definir os atributos de uma classe como privados, o que significa que esses dados não podem ser acessados direta de fora da classe. Isso é essencial para evitar que dados sensíveis ou críticos sejam manipulados incorreta ou indevidamente. Por exemplo, imagine uma classe ContaBancaria que tenha um atributo de saldo. Ao tornar o saldo privado, há um impedimento para qualquer código externo modificar o saldo diretamente, o que poderia levar a resultados indesejados.

Para permitir que os objetos dessa classe sejam utilizados, podemos definir métodos públicos que atuam como interface para interações externas. Esses métodos são responsáveis por acessar ou modificar os atributos privados de forma controlada e segura. No caso da classe ContaBancaria, podemos criar métodos públicos como **depositar** e **sacar**, que permitem aos usuários da classe interagir com o saldo de maneira controlada e segura. Esses métodos são chamados de interface pública.

O encapsulamento oferece controle sobre como os dados são acessados e modificados. Isso significa que podemos impor regras e validações para garantir que os dados permaneçam em estado consistente. No exemplo da ContaBancaria, o método sacar pode verificar se o saldo é suficiente antes de permitir uma retirada; se o saldo for insuficiente, ele pode gerar um erro ou uma exceção, garantindo que o saldo nunca fique negativo.

Outra vantagem do encapsulamento é facilitar a manutenção do código. Como os detalhes internos de uma classe são encapsulados e não expostos diretamente, podemos fazer alterações internas, como a estrutura de dados usada para armazenar os atributos, sem afetar o código externo que usa a classe. Isso é conhecido como **separação de preocupações** e ajuda a isolar mudanças internas para evitar efeitos colaterais não desejados. O encapsulamento também permite a evolução e o aprimoramento das classes com mais facilidade. Se for necessário adicionar novos recursos ou validar dados de maneira diferente, podemos fazer isso dentro dos métodos públicos sem afetar o código que usa a classe.

O encapsulamento também reutiliza o código. É possível criar uma classe bem encapsulada e, em seguida, usá-la em diferentes partes do programa sem se preocupar com os detalhes internos, economizando tempo e esforço, pois não será preciso reescrever o mesmo código repetidamente. Ao ocultar os detalhes internos de um objeto, o encapsulamento ajuda a proteger o estado do objeto contra alterações não autorizadas, o que é importante para garantir a integridade dos dados e a segurança do programa.

Agora confira a figura 3:

```
1.  public class Caneta
2.  {
3.     public string Cor { get; set; }          // Atributo público
4.     private int Carga { get; set; }          // Atributo privado
5.
6.     public Caneta(string cor, int carga)     // Método Construtor
7.     {  Cor = cor; Carga = carga; }
8.
9.     public void Escrever(string texto)       // Método público para escrever
10.    {
11.       if (Carga > 0)
12.       {
13.          Console.WriteLine($"Escrevendo '{texto}' com uma caneta {Cor}.");
14.          Carga--;
15.       }
16.       else
17.       {
18.              Console.WriteLine("A caneta está sem carga.");
19.       }
20.    }
21.
22.    public void Recarregar()                 // Método público para recarregar
23.    {
24.       Carga = 100;
25.       Console.WriteLine("Caneta recarregada.");
26.    }
27.
28.    public int GetCarga()            // Método público para obter a carga
29.    {
30.       return Carga;
31.    }
32. }
```

Figura 3 – Código-fonte escrito na linguagem C# que encapsula os atributos de uma caneta

O atributo Cor foi definido como público (no C# usamos a palavra-chave **public**), já o atributo Carga foi definido como privado (no C# usamos a palavra-chave **private**). Isso significa que a informação da carga somente é acessível, ou seja, apenas está visível ou disponível para a própria classe; outras classes não podem modificar a informação de carga. Por se tratar de um número inteiro (no C# usamos a palavra-chave **int**), o atributo carga abstrai a quantidade de carga da caneta. Desse modo, se a caneta tiver o valor 0 (zero) no atributo carga, ela está sem carga; se tiver o valor 100 (cem), estará com carga completa.

Observe atentamente as linhas 9 a 20, pois é o trecho de código que abstrai a escrita da caneta. Dessa vez nosso foco não será a abstração, e sim o encapsulamento. Suponha que o programador crie, no futuro, uma caneta com o valor "azul" atribuído à Cor. O método Escrever, ao ser acionado e receber um texto, por exemplo "Bom dia!", exibirá a mensagem "Escrevendo 'Bom dia!' com uma caneta azul.". Além de mostrar essa frase, o método Escrever decrementará a carga da caneta em uma unidade (linha 14), sinalizando corretamente que, ao escrever, a caneta perdeu um pouco (uma unidade) de sua carga. Toda vez que o método é invocado, uma unidade de carga é retirada da caneta; note que somente

o método Escrever, pertencente à classe Caneta, pode alterar o valor da Carga. Eis aí o encapsulamento aplicado, garantindo que o valor da carga somente será reduzido após a escrita. Nenhuma outra classe externa à caneta poderá mudar a carga. Mesmo se o programador criasse uma classe Caderno, ele não conseguiria diminuir a carga da caneta diretamente; precisaria obrigatoriamente acionar o método Escrever da caneta. O método escrever ainda emite a mensagem "A caneta está sem carga." caso o método seja acionado em uma caneta com o valor zero no atributo carga.

Em resumo, encapsulamento é um conceito fundamental na POO, pois envolve a proteção dos detalhes internos de um objeto, limitando o acesso direto aos seus atributos e métodos. Isso proporciona controle, segurança, reutilização de código e facilita a manutenção e evolução do software. É uma prática recomendada para escrever código eficiente, organizado e seguro em linguagens de POO.

Herança

Permite que uma classe (ou objeto) herde características (atributos e métodos) de outra classe, promovendo a reutilização de código e possibilitando criar uma nova classe baseada em outra, estendendo ou modificando seu comportamento. A classe derivada herda os atributos e métodos da classe-base e pode adicionar funcionalidades ou substituir métodos existentes.

A herança também estabelece uma relação do tipo "é um" entre a classe-base (ou superclasse) e a classe derivada (ou subclasse). Dentre os benefícios da herança, podemos citar também organização, modularidade (classes semelhantes podem ser agrupadas sob uma superclasse comum, promovendo um design mais limpo) e extensibilidade (classes derivadas podem estender ou modificar o comportamento herdado da superclasse, adaptando-o às suas necessidades).

Vamos ilustrar esse conceito com a figura 4.

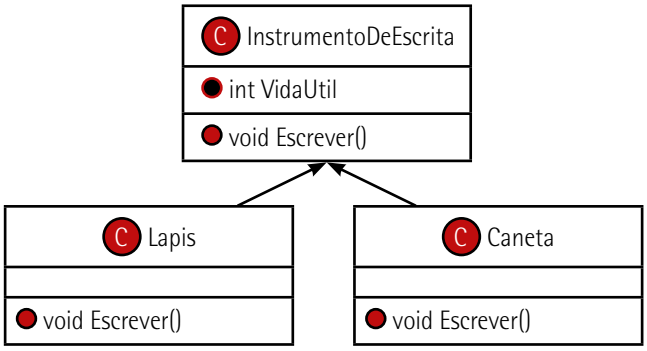

Figura 4 – Diagrama de classes que demonstra o conceito de herança

Lápis e caneta são subclasses que derivam da classe chamada instrumento de escrita. Em UML os retângulos denotam as classes, e as subdivisões de cada retângulo representam respectivamente o nome da classe, seus atributos e métodos (Booch; Rumbaugh; Jacobson, 2005). As setas vazadas (ponta triangular) indicam a herança, a classe InstrumentoDeEscrita tem um atributo do tipo inteiro (int) chamado VidaUtil e um método Escrever, e as classes Lapis e Caneta herdam dessa superclasse (ou classe-pai).

O conceito de herança não é difícil, mas à primeira vista pode parecer um pouco confuso de ser escrito na linguagem de programação. A figura 4, se implantada na linguagem C#, pode ter um código-fonte conforme a figura 5.

```
1.  public class InstrumentoDeEscrita        // Classe base ou superclasse
2.  {
3.    public int VidaUtil { get; set; }// em porcentagem
4.
5.    public InstrumentoDeEscrita(int vidaUtil)
6.    {
7.      this.VidaUtil = vidaUtil;
8.    }
9.
10.   public virtual void Escrever()
11.   {
12.     if (VidaUtil > 0)
13.     {
14.         VidaUtil -= 10;       // reduz a vida útil em 10% a cada vez
que escreve
15.         Console.WriteLine("Está escrevendo com o instrumento. Vida
útil agora: " + VidaUtil + "%");
16.     }
17.     else
18.     {
19.         Console.WriteLine("Instrumento sem vida útil para
escrever.");
20.     }
21.   }
22. }
23.
24. // Classe Lapis herdando de InstrumentoDeEscrita
25. public class Lapis : InstrumentoDeEscrita
26. {
27.   public Lapis(int vidaUtil) : base(vidaUtil) { }
28.
29.   public override void Escrever()
30.   {
31.     // Lógica específica para lapis (se necessário)
32.     base.Escrever();      // Chama o método da superclasse
33.   }
34. }
35.
36. // Classe Caneta herdando de InstrumentoDeEscrita
37. public class Caneta : InstrumentoDeEscrita
38. {
39.   public Caneta(int vidaUtil) : base(vidaUtil) { }
40.
41.   public override void Escrever()
42.   {
43.     // Lógica específica para caneta (se necessário)
44.     base.Escrever();      // Chama o método da superclasse
45.   }
46. }
```

Figura 5 – Código-fonte escrito na linguagem C# que demonstra o conceito de herança

Novamente, não se preocupe ainda com os termos em inglês que aparecem no código-fonte nem com a sintaxe do C#, tampouco com a plataforma usada para escrevê-lo, se funciona adequadamente ou se precisa de complementos. Vamos nos concentrar nos detalhes da herança.

Na linha 1 foi definida uma classe-base chamada InstrumentoDeEscrita, e nas linhas 25 e 37 foram definidas duas subclasses, Lapis e Caneta; ambas herdam os atributos e métodos da classe-base. As linhas de 1 a 21 detalham um instrumento de escrita e, dessa forma, servem tanto para o lápis quanto para a caneta. A linha 3 define o atributo VidaUtil, que na caneta representa a quantidade de carga (tinta) no tubo, e no lápis representa a quantidade de grafite que ele contém. Portanto, VidaUtil não será medida em dias ou meses, e sim em um percentual de carga ou grafite que se degradam (ou desgastam) à medida que o instrumento é usado na sua função principal que é a escrita. Por esse mesmo motivo, o método Escrever faz parte do instrumento de escrita e basicamente escreve a mensagem e faz o decaimento (ou redução) da VidaUtil. A linha 14 define arbitrariamente uma redução de 10 unidades de VidaUtil a cada escrita. Observe também, que não há um número inicial de VidaUtil: isso será definido no momento de criação de cada lápis e de cada caneta. Podemos ter, por exemplo, um lápis com VidaUtil de 100 (e neste caso ele somente conseguiria escrever dez vezes) e uma caneta com VidaUtil de 70 (que apenas conseguiria escrever sete vezes).

Lapis e Caneta herdam a propriedade VidaUtil e o método Escrever da classe InstrumentoDeEscrita. Ambas as classes-filhas são capazes de redefinir (ou override) o comportamento do método Escrever caso queiram adicionar lógica específica. O uso da palavra-chave **virtual** na superclasse (linha 10) indica que o método pode ser sobrescrito em subclasses. Quando uma classe derivada decide sobrescrever esse método, ela usa a palavra-chave **override** (linhas 29 e 41). Veremos essa sintaxe com mais detalhes de modo oportuno.

Existem duas formas de herança: simples e múltipla. Algumas linguagens, como C# e Java, suportam apenas herança simples (uma classe pode herdar apenas de uma superclasse). Outras linguagens, como C++, permitem herança múltipla, na qual uma classe pode herdar de várias superclasses. Existe também um princípio chamado **substituição de Liskov**, segundo o qual objetos de uma superclasse devem ser capazes de ser substituídos por objetos de uma subclasse sem afetar a correção do programa (Liskov; Wing, 1993).

Polimorfismo

O termo deriva das palavras gregas **poli** (muitos) e **morph** (formas), e pode ser entendido como a capacidade de um objeto assumir muitas formas. Em termos de programação, refere-se à habilidade de objetos de classes diferentes serem tratados como instâncias de uma mesma classe, principalmente por meio de herança e interfaces.

O polimorfismo traz benefícios como flexibilidade, extensibilidade e manutenibilidade do código-fonte. A flexibilidade permite que um sistema evolua facilmente; se novas classes forem adicionadas, elas podem simplesmente ser integradas ao sistema existente, com modificações mínimas. Já a extensibilidade significa que novas classes podem ser adicionadas com pouco ou nenhuma modificação nas classes

existentes, tornando o sistema mais extensível. Por fim, a manutenibilidade diz respeito à contribuição do polimorfismo para um código mais limpo e organizado, facilitando a manutenção.

No exemplo da figura 6, Lapis e Caneta são subclasses de InstrumentoDeEscrita. O método ExecutarEscrita (linha 24) é um exemplo de polimorfismo em ação: ele aceita qualquer objeto que seja um InstrumentoDeEscrita (ou seja, um Lapis ou uma Caneta) e chama o método Escrever desse objeto.

```
1.  public abstract class InstrumentoDeEscrita
2.  {
3.    public abstract void Escrever();
4.  }
5.
6.  public class Lapis : InstrumentoDeEscrita
7.  {
8.    public override void Escrever()
9.    {
10.     Console.WriteLine("Escrevendo com o lápis!");
11.   }
12. }
13.
14. public class Caneta : InstrumentoDeEscrita
15. {
16.   public override void Escrever()
17.   {
18.     Console.WriteLine("Escrevendo com a caneta!");
19.   }
20. }
21.
22. public class MainClass
23. {
24.   public static void ExecutarEscrita(InstrumentoDeEscrita
instrumento)
25.   {
26.     instrumento.Escrever();
27.   }
28.
29.   public static void Main(string[] args)
30.   {
31.     InstrumentoDeEscrita lapis = new Lapis();
32.     InstrumentoDeEscrita caneta = new Caneta();
33.
34.     ExecutarEscrita(lapis); // Output: Escrevendo com o lápis!
35.     ExecutarEscrita(caneta); // Output: Escrevendo com a caneta!
36.   }
37. }
```

Figura 6 – Código-fonte escrito na linguagem C# que demonstra o conceito de polimorfismo

Na linha 34, quando ExecutarEscrita(lapis) é chamado, o método Escrever da classe Lapis é executado, e na linha 35, quando ExecutarEscrita(caneta) é chamado, o método Escrever da classe Caneta é executado. Essa é a essência do polimorfismo: tratar diferentes objetos de forma consistente, sem necessariamente saber de que tipo são.

Existem três tipos de polimorfismo:

- **De subtipos (ou de inclusão)**: é o tipo mais comum em linguagens orientadas a objetos, como Java, C# e Python. Ocorre quando uma classe-filha é tratada como instância de sua classe-pai. Isto é, um objeto da subclasse pode ser usado em lugar de um objeto da superclasse.

- **De sobrecarga**: refere-se à capacidade de várias funções terem o mesmo nome, mas com assinaturas diferentes. Isso significa que você pode ter múltiplos métodos com o mesmo nome na mesma classe, contanto que os parâmetros sejam diferentes.

- **Paramétrico**: geralmente associado a linguagens de programação genéricas, onde um tipo de dado é definido de maneira geral e depois especificado durante o uso.

Como vimos, polimorfismo é uma poderosa ferramenta na POO, pois permite maior reutilização de código, maior abstração e uma forma de integrar novas classes em sistemas existentes com pouco esforço. Também possibilita que objetos de diferentes classes sejam tratados como se fossem do mesmo tipo, o que é fundamental para construir sistemas robustos, extensíveis e fáceis de manter.

Lembrete

A POO se fundamenta em quatro pilares essenciais que interagem para criar software modular e reutilizável: encapsulamento, herança, polimorfismo e abstração. **Encapsulamento** refere-se ao agrupamento de dados e funções que operam nesses dados dentro de uma única unidade, chamada objeto, protegendo a integridade dos dados ao restringir o acesso direto a eles e permitindo apenas a interação por métodos bem definidos, promovendo assim a segurança e a modularidade do software – característica que transforma os objetos em "caixas pretas", nas quais a implementação interna é oculta, e apenas a interface – ou seja, a maneira como os outros objetos interagem com ele – é visível.

O próximo pilar, **herança**, é um mecanismo que permite a uma nova classe herdar propriedades e comportamentos de uma classe existente, facilitando a reutilização de código e a criação de categorias hierárquicas. Em vez de reescrever funções ou características comuns, os desenvolvedores podem criar subclasses que herdam atributos e métodos de uma superclasse, e ainda têm a flexibilidade de introduzir ou modificar características conforme necessário. Isso promove uma estruturação lógica e uma forma eficiente de propagar mudanças, já que a alteração na superclasse reflete automaticamente em suas subclasses.

Polimorfismo, o terceiro pilar, refere-se à capacidade dos objetos de assumir múltiplas formas. Em termos práticos, isso significa que diferentes classes podem ser tratadas como instâncias de uma mesma classe devido à

herança. Isso é útil em estruturas de código onde a especificidade exata do objeto pode não ser conhecida, permitindo flexibilidade e extensibilidade.

Finalmente, **abstração** é a ideia de esconder detalhes complexos e mostrar apenas as características essenciais, ajudando os programadores a lidar com sistemas complexos ao focar níveis mais altos de interação. Ela permite aos desenvolvedores criar componentes com interfaces claras, escondendo a complexidade por trás de operações e processos. Ao trabalhar com abstrações, os programadores podem se concentrar na lógica de alto nível, confiando que os detalhes internos funcionam conforme especificado.

Em resumo, esses quatro pilares da POO proporcionam uma base sólida para criar sistemas robustos, mantendo a complexidade sob controle e promovendo a reutilização de código, tornando o desenvolvimento de software mais intuitivo e eficiente.

1.4 Breve histórico

A POO é indiscutivelmente um dos paradigmas mais influentes na história da computação. A gênese e os primeiros passos do conceito de objetos em programação emergiram nas décadas anteriores a 1960, período que pôs as fundações para o que viria a se tornar a POO. Nos anos 1950, o cenário da programação era dominado por linguagens de baixo nível e por paradigmas procedurais. Entretanto, começaram a surgir as primeiras sementes do que seria a orientação a objetos.

Em 1960, o ALGOL 58 introduziu blocos de código, um prenúncio das futuras classes e métodos. No entanto, foi com o ALGOL 60 que a ideia de registro surgiu, permitindo agrupar dados e operações em uma única unidade. Os primórdios da POO não surgiram aleatoriamente; eles foram o produto de uma série de inovações, experimentações e do reconhecimento crescente da necessidade de melhores abstrações na programação. As ideias e linguagens dessa época não eram "orientadas a objetos" no sentido moderno, mas sem elas a POO talvez não teria evoluído da maneira como a conhecemos hoje.

A primeira linguagem a introduzir algumas ideias que seriam mais tarde associadas à POO foi LISP (McCarthy, 1960), concebida em 1958 por John McCarthy enquanto estava no Massachusetts Institute of Technology (MIT). Embora LISP seja mais associada à programação funcional, sua capacidade de manipular símbolos e listas encorajam uma abordagem mais abstrata e modular à programação. Na mesma época, Grace Hopper liderou o desenvolvimento da COBOL, uma das primeiras linguagens a adotar um paradigma que se afasta do estilo de máquina de baixo nível e permite uma forma mais humanamente legível de programação (Sammet, 1978). Apesar de não ser orientada a objetos, ela demonstrou a importância e a possibilidade de abstrair os detalhes de hardware e se concentrar na lógica do programa.

O conceito central da POO é o objeto. Durante os anos 1950, houve uma crescente conscientização sobre a eficácia de visualizar programas como coleções de "blocos de construção" (Parnas, 1972). A ideia era que, ao compor programas a partir de componentes independentes e reutilizáveis (ou seja, objetos),

a complexidade poderia ser melhor gerenciada e a eficiência melhorada. Além dos desenvolvimentos puramente computacionais, havia uma forte influência da teoria dos sistemas e da cibernética (Ashby, 1956) durante o período. Essas disciplinas exploraram a ideia de sistemas compostos de componentes interativos e interdependentes – uma abordagem que se alinhava bem com o conceito emergente de objetos em programação.

Ainda que a Simula – primeira linguagem de programação verdadeiramente orientada a objetos – tenha sido criada nos anos 1960, o trabalho preliminar começou na década de 1950. Ole-Johan Dahl e Kristen Nygaard, do Norwegian Computing Center (NCC), em Oslo, estavam trabalhando em simulações e começaram a desenvolvê-la para lidar com as complexidades desse trabalho (Dahl; Nygaard, 1966), introduzindo conceitos como classes, instâncias, herança e subclasse. Originalmente concebida como uma extensão da linguagem ALGOL, sua intenção era facilitar a simulação de eventos do mundo real, como processos de negócios e sistemas de tráfego.

A Simula introduziu o conceito de classe como um protótipo para criar instâncias (objetos). Classes encapsulam tanto dados (atributos) quanto procedimentos (métodos) que operam nesses dados. Também foi pioneira no conceito de herança, onde uma subclasse poderia herdar propriedades e comportamentos de uma superclasse, permitindo a reutilização de código e a criação de hierarquias de classe. Além da Simula, outras linguagens começaram a explorar a herança e o polimorfismo, permitindo que objetos herdassem propriedades e comportamentos e sobrescrevessem ou expandissem funcionalidades de suas classes-pai (Meyer, 1982).

Já o encapsulamento, outro pilar da POO, começou a ganhar destaque também nessa época. Ele promove a ideia de que os dados de um objeto devem ser ocultados de outros objetos e somente podem ser acessados por métodos predeterminados. O uso da abstração, permitindo que os programadores vejam apenas a interface relevante de um objeto, tornou-se crucial. Esse conceito abstrai os detalhes de implementação, tornando os programas mais gerenciáveis e eficientes (Dijkstra, 1959). Outro marco crucial foi a publicação de "Go to statement considered harmful" por Edsger Dijkstra (1968), que incentivou o uso de estruturas de controle mais estritas, alinhando-se com a abordagem da POO em encapsular a complexidade (Dijkstra, 1968).

A Xerox Palo Alto Research Center (Parc) tornou-se o berço da revolução da POO com o desenvolvimento da Smalltalk (Kay, 1977), que lhe permitiu um salto evolutivo ao incorporar todos os seus princípios fundamentais de maneira pura. Sob a orientação de Alan Kay (que cunhou o termo programação orientada a objetos), a linguagem introduziu uma visão puramente orientada a objetos da programação, influenciando fortemente linguagens futuras. Tudo na Smalltalk – desde números até classes e blocos de código – é tratado como objeto, o que uniformizou seu design, e a mesma abordagem orientada a objetos poderia ser aplicada a qualquer entidade no sistema. Em vez de chamar métodos diretamente, na Smalltalk os objetos se comunicam por mensagem. Isso originou um polimorfismo mais flexível, onde diferentes objetos poderiam responder à mesma mensagem de maneiras distintas. Smalltalk foi notável não apenas por sua linguagem, mas também por seu ambiente de desenvolvimento integrado. Esse ambiente oferecia uma interface gráfica onde os programadores podiam interagir diretamente com objetos, modificando-os e observando suas respostas em tempo real.

Ambas as contribuições – de Dahl e Nygaard com Simula, e de Kay com Smalltalk – foram fundamentais para moldar o paradigma da POO. Seus conceitos e práticas influenciaram profundamente o desenvolvimento de software e a concepção de muitas linguagens de programação que vieram depois. Durante a década de 1970, também surgiu a ideia de armazenar objetos diretamente em bancos de dados. Embora os bancos de dados orientados a objetos não tenham ganhado tração comercial imediatamente, o conceito permaneceu crucial para a evolução da POO. Na época os ambientes de desenvolvimento também começaram a se adaptar, surgindo os *integrated development environments* (IDEs – em português, ambientes de desenvolvimento integrado) e ferramentas para facilitar a POO.

A década de 1980 testemunhou a popularização da POO. A linguagem C++, desenvolvida por Bjarne Stroustrup, expandiu o conceito de objetos com a adição de classes e a combinação de características da linguagem C, e em 1983 James Gosling iniciou o desenvolvimento da linguagem Oak, posteriormente evoluindo para o Java, que por sua vez trouxe a portabilidade de código e reforçou os princípios da POO.

Já nos anos 1990 a criação de padrões de design orientado a objetos se popularizou. O livro *Design patterns: elements of reusable object-oriented software* (1994), de Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides (conhecidos como Gang of Four), introduziu 23 padrões de design amplamente utilizados na indústria. Na mesma época, a linguagem Python, criada por Guido van Rossum em 1991, adotou os princípios da POO, enfatizando a legibilidade do código. Python se tornou uma das linguagens mais populares e influentes da Era Moderna.

A história da POO continua com o advento de linguagens como C#, Ruby e Kotlin, cada uma trazendo suas próprias inovações e abordagens à POO. O surgimento de estruturas e bibliotecas como o .NET Framework, Ruby on Rails e Spring Framework revolucionou o desenvolvimento orientado a objetos, simplificando tarefas complexas. Além disso, a POO se integrou a outras áreas, como o desenvolvimento web e móvel, expandindo seu alcance e relevância em todas as esferas da computação.

Podemos resumir a evolução da POO como uma jornada de inovação e colaboração, tendo como principais fases:

- **1950-1960**: busca por um novo paradigma não estruturado.
- **1960-1970**: raízes do novo paradigma.
- **1970-1980**: surgimento do conceito.
- **1980-1990**: popularização e expansão.
- **1990-2000**: proliferação de padrões e práticas.
- **2000-presente**: evolução contínua.

A história da POO é uma narrativa fascinante que se estende por décadas, marcada por inovações, debates e contribuições notáveis de indivíduos e comunidades de todo o mundo.

Saiba mais

Existem muitos detalhes interessantes na história da evolução da computação, bem como da POO. Para mais detalhes, recomendamos a seguinte leitura:

FONSECA FILHO, C. *História da computação*: o caminho do pensamento e da tecnologia. Porto Alegre: EDIPUCRS, 2007.

2 ORIENTAÇÃO A OBJETOS EM C#

Compreendido o contexto e a relevância da criação e adoção do paradigma orientado a objetos, e assimilados seus conceitos-chave – como classes, objetos, abstração, encapsulamento, herança e polimorfismo –, é possível utilizar uma linguagem prática para implementar soluções.

Em outras palavras, tendo entendido os conceitos, é possível iniciar a programação propriamente dita para produzir artefatos de software (programas), e este capítulo aborda os primeiros passos na programação com a linguagem C#. Após a leitura atenta e a prática dos exercícios aqui apresentados, você será capaz de implementar os conceitos aprendidos no tópico 1 em linguagem C#.

Para executar códigos-fonte, é necessário compilá-los. Recomendamos que esse processo seja feito em ambientes de desenvolvimento, como o Visual Studio, da empresa Microsoft. Como tais ambientes requerem a instalação e configuração de ferramentas específicas, sugeriremos também uma estratégia mais simples, os compiladores online, pois são versões simplificadas dos robustos ambientes de desenvolvimento e, por esse motivo, facilitam a tarefa de iniciantes de programação.

Para encontrar um compilador online é possível digitar "c# online compiler" no seu buscador web. Existem diversos compiladores, mas por questões de privacidade de dados vale a pena encontrar algum gratuito e sem a necessidade de criar uma conta para utilizá-lo. A maioria deles tem o mesmo estilo de interface gráfica: à esquerda da tela você digita o código-fonte; à direita você vê o resultado após a compilação (feita ao apertar um botão como "run", "compile", "execute" etc., geralmente no centro da tela, na parte superior). Outra vantagem desses compiladores é o fato de executarem no navegador e poderem ser acessados por smartphones com acesso à internet.

2.1 Criando classes em C#

Em C#, classe é uma estrutura fundamental da linguagem que define um tipo de objeto. É uma maneira de organizar e encapsular dados (atributos) e comportamentos (métodos) relacionados em um único bloco de código. Classes são a base da POO em C# e são usadas para criar objetos, que são instâncias dessa classe.

A declaração de uma classe em C# segue o formato geral ilustrado na figura 7:

```
1.  modificador_de_acesso class NomeClasse
2.  {
3.     // Atributos (variáveis de instância)
4.     modificador_de_acesso tipo_de_dado atributo1;
5.     modificador_de_acesso tipo_de_dado atributo2;
6.     // ...
7.     // Construtor (opcional)
8.     modificador_de_acesso NomeClasse(parâmetros)
9.     {
10.       // Inicialização dos atributos
11.    }
12.    // Métodos (funções)
13.    modificador_de_acesso tipo_de_retorno NomeMetodo(parâmetros)
14.    {
15.       // Corpo do método
16.    }
17. }
```

Figura 7 – Formato geral para declaração de uma classe

Os principais elementos nessa declaração são:

- **modificador_de_acesso**: define o nível de acesso à classe e seus membros. Os modificadores mais comuns são public, private, protected e internal.

- **NomeClasse**: nome da classe que você está declarando. Deve começar com uma letra e seguir convenções de nomenclatura. Falaremos sobre isso com mais detalhes em breve.

- **Atributos (variáveis de instância)**: pertencem a cada objeto criado a partir da classe. Você pode especificar seu tipo de dados e modificador de acesso. Representam as características dos objetos da classe.

- **Construtor (opcional)**: método especial que inicializa objetos quando criados a partir da classe. Uma classe pode ter vários construtores com diferentes parâmetros.

- **Métodos (funções)**: definem os comportamentos associados à classe e podem aceitar parâmetros e retornar valores. Os métodos encapsulam a funcionalidade que os objetos da classe podem executar.

O formato geral pode parecer um pouco confuso, por isso é importante avaliarmos um exemplo prático. Confira a figura 8:

```
1.  public class Pessoa
2.  {
3.     // Atributos
4.     public string Nome;
5.     public int Idade;
6.
7.     // Construtor
8.     public Pessoa(string nome, int idade)
9.     {
10.       Nome = nome;
11.       Idade = idade;
12.    }
13.
14.    // Método
15.    public void Apresentar()
16.    {
17.       Console.WriteLine($"Olá, meu nome é {Nome} e tenho {Idade}
anos.");
18.    }
19. }
```

Figura 8 – Exemplo da criação de uma classe chamada Pessoa

A palavra **public** é um modificador_de_acesso usado nas linhas 1 (deixando público o acesso à classe), 4 e 5 (deixando público o acesso aos atributos da classe), 8 (deixando público o acesso ao método construtor da classe) e 15 (deixando público o acesso a um método da classe, chamado Apresentar).

Também é importante observar que a classe tem seu conteúdo delimitado pelo símbolo de abre-chaves (linha 2) e fecha-chaves (linha 19); tudo que está escrito entre essas linhas é parte integrante da classe, faz parte de sua definição. No seu conteúdo interno foram definidos dois atributos ou variáveis de instância (linhas 4 e 5) e dois métodos (linhas 8 a 12 e 15 a 18, respectivamente).

Atributos contêm o modificador_de_acesso e o tipo_de_dado (cadeia de caracteres e inteiro, respectivamente) e nomes que o programador define. Geralmente a escolha de nomes preconiza palavras fáceis de lembrar: em vez de escrever "A32WZ15" ou algo do tipo para se referir à idade da pessoa, é melhor escrever Idade mesmo. Isso também vale para nomes de classes e métodos. Uma boa programação pressupõe um código-fonte de boa legibilidade.

O primeiro método (que tem o mesmo nome da classe) é especial: é o Construtor da classe. É composto do modificador_de_acesso, do nome da classe e dos parâmetros colocados entre parênteses e separados por vírgula. Note que cada um dos parâmetros possui o tipo_de_dado e um nome. As linhas 9 a 12 representam o conteúdo interno do método e têm instruções em C# a serem executadas somente quando ele for invocado. Convém lembrar que a declaração de métodos construtores é opcional.

O segundo método não é um construtor e tem um tipo_de_retorno antes do nome Apresentar. Similarmente, as linhas 16 a 18 representam o conteúdo do método, encapsulando seu funcionamento.

Após a descrição detalhada do exemplo, algumas regras para especificar o nome das classes em C#:

- **Identificadores válidos**: o nome de classes deve começar com uma letra ou sublinhado (_) e pode conter letras, dígitos e sublinhados. Não pode começar com dígito.

- **Case sensitive**: os nomes de classe são case sensitive, ou seja, maiúsculas e minúsculas são distintas. Por exemplo, MinhaClasse e minhaclasse são nomes de classe diferentes.

- **Palavras-chave**: não se pode usar palavras-chave da linguagem C# como nome de classe. Por exemplo, você não pode nomear uma classe como int, string, public etc.

- **Convenções de nomenclatura**: embora não haja um limite estrito para o número de caracteres em um nome de classe, é prática comum seguir convenções de nomenclatura para manter o código legível. A convenção de nomenclatura geralmente usada é o "PascalCase", onde cada palavra no nome da classe começa com uma letra maiúscula e não contém espaços nem caracteres especiais. Por exemplo, MinhaClasse, ClienteVIP etc.

Declarada a classe Pessoa, é importante lembrar que ela não criou nenhum objeto do tipo Pessoa. A declaração é apenas a descrição de sua estrutura, de sua lógica, de seu funcionamento; não é o objeto em si.

Outro exemplo de criação de classe: na mitologia grega, deidades são divindades ou deuses e deusas venerados pela humanidade. Eles fazem parte de um panteão que governa diferentes aspectos da vida, natureza e cosmos. Muitas delas são antropomórficas, ou sejam, têm características humanas – como emoções, personalidades e formas físicas –, mas também formas animais ou naturais.

O quadro 1 mostra algumas deidades conhecidas da mitologia:

Quadro 1 – Exemplos de deidades

Nome	Domínio
Zeus	Rei dos deuses, trovão e relâmpago
Atena	Sabedoria, estratégia militar e justiça
Afrodite	Amor, beleza e paixão
Apolo	Artes, música, luz e Sol
Poseidon	Mares, oceanos e terremotos
Hades	Submundo e mortos
Deméter	Agricultura e fertilidade
Hermes	Mensageiro dos deuses, comércio e viajantes
Dionísio	Vinho, teatro e festas
Héstia	Lar e lareira

A figura 9 apresenta um exemplo mais simples de criação de uma classe chamada Deidade, com duas propriedades (que são mecanismos de acesso aos campos de uma classe) e sem métodos declarados.

```
1.  public class Deidade
2.  {
3.    public string Nome { get; set; }
4.    public string Dominio { get; set; }
5.  }
```

Figura 9 – Classe Deidade: exemplo simples de criação de classe

É possível colocar mais detalhes nessa classe. A figura 10 inclui, além das propriedades Nome e Domínio, um método construtor e um método chamado InvocarPoder(). O construtor da classe aceita parâmetros para inicializar suas propriedades, ou seja, inicializa objetos do tipo Deidade com nomes e domínios escolhidos pelo criador do objeto.

```
1.  public class Deidade
2.  {
3.    public string Nome { get; set; }
4.    public string Dominio { get; set; }
5.
6.    public Deidade(string nome, string dominio)
7.    {
8.      Nome = nome;
9.      Dominio = dominio;
10.   }
11.
12.   public void InvocarPoder()
13.   {
14.     Console.WriteLine($"Eu, {Nome}, invoco o poder do {Dominio}!");
15.   }
16. }
```

Figura 10 – Classe Deidade: exemplo mais detalhado

Lembre-se que os modificadores de acesso em C# definem o escopo de visibilidade e acessibilidade de membros de uma classe, estrutura, interface ou enumeração. Eles determinam o nível de acesso de outros tipos ou membros a esses membros. A correta utilização dos modificadores de acesso é fundamental para garantir o encapsulamento, um dos pilares da POO.

A seguir, os principais modificadores de acesso da linguagem.

2.1.1 Private

Restringe o acesso ao membro somente ao tipo que o contém. Campos de uma classe ou estrutura geralmente são declarados como private para impedir acesso direto de fora da classe, protegendo a integridade dos dados e permitindo que a classe controle como seus dados são acessados ou modificados através de métodos ou propriedades.

A figura 11 ilustra um exemplo de classe com atributo privado:

```
1.  class Exemplo
2.  {
3.     private int numeroPrivado;
4.  }
```

Figura 11 – Exemplo de modificador private

2.1.2 Public

Permite que um membro seja acessado de qualquer outro código no mesmo assembly ou em outro assembly que o referencie. Costuma ser usado para permitir acesso irrestrito a um membro de um tipo. Propriedades e métodos que precisam ser acessíveis a partir de qualquer parte do código geralmente são marcados como public.

A figura 12 ilustra um exemplo de classe pública com método também público:

```
1.  public class Exemplo
2.  {
3.     public void MetodoPublico() { }
4.  }
```

Figura 12 – Exemplo de modificador public

2.1.3 Protected

Permite que um membro seja acessado apenas pela classe que o contém e por qualquer classe derivada. Costuma ser usado para subclasses poderem acessar o membro, mas não qualquer outro código externo. Fornece funcionalidades-base que as subclasses podem usar ou estender. Falaremos de herança em C# no tópico 2.10.

A figura 13 ilustra um exemplo de classe com atributo protegido (a classe Derivada da classe Base tem acesso ao atributo).

```
1.  class Base
2.  {
3.     protected int numeroProtegido;
4.  }
5.  class Derivada : Base
6.  {
7.     public void AcessarNumero()
8.     {
9.        numeroProtegido = 10;  // Permitido, pois Derivada herda de Base
10.    }
11. }
```

Figura 13 – Exemplo de modificador protected

2.1.4 Internal

Permite que um membro seja acessado por qualquer código no mesmo assembly, mas não de um assembly diferente. Costuma ser usado quando se deseja expor membros dentro de um assembly, mas esconder detalhes de implementação de consumidores em assemblies externos – por exemplo, classes e métodos usados internamente em uma biblioteca mas não destinados a ser usados por quem referencia essa biblioteca.

A classe da figura 14 somente será acessada no mesmo assembly:

```
1.   internal class ClasseInterna { }
```

Figura 14 – Exemplo de modificador internal

2.1.5 Protected Internal

Modificador combinado, o protected internal pode ser acessado por qualquer código no mesmo assembly ou por uma classe derivada em outro assembly. Costuma ser usado quando desejamos que um membro seja visível para todas as classes no mesmo assembly, bem como classes derivadas em assemblies externos.

A figura 15 ilustra o uso desse modificador em um atributo:

```
1.  class Base
2.  {
3.    protected internal int numeroProtegidoInterno;
4.  }
```

Figura 15 – Exemplo de modificador protected internal

2.1.6 Private protected (introduzido no C# 7.2)

Um membro private protected é acessível somente em seu assembly, e apenas por tipos derivados. Costuma ser usado quando desejamos que um membro seja visível para subclasses no mesmo assembly, mas não para subclasses em assemblies externos ou qualquer outra classe no mesmo assembly.

A figura 16 ilustra o uso desse modificador em um atributo:

```
1.  class Base
2.  {
3.    private protected int numeroPrivadoProtegido;
4.  }
```

Figura 16 – Exemplo de modificador private protected

De maneira geral, os dois primeiros modificadores são os mais usados. O terceiro é usado casualmente, e os demais são usados com menor frequência.

Observação

Em .NET, assembly é uma unidade fundamental de implantação, versão, reutilização, ativação de segurança e compartilhamento de código. É uma coleção de tipos e recursos construídos e compilados para funcionar juntos de forma lógica. Em termos mais práticos, assembly é geralmente um arquivo .dll (biblioteca de ligação dinâmica) ou .exe (executável), mas pode abranger múltiplos arquivos físicos.

2.2 Atributos, campos e constantes

Em POO, **atributos** são as características ou propriedades dos objetos, representam os dados associados a uma classe e descrevem o estado de um objeto. São variáveis de instância dentro da classe e têm diferentes tipos de dados, como números, strings ou outros objetos (as linhas 4 e 5 da figura 7 demonstram como declarar um atributo). Atributos encapsulam o estado de um objeto e podem ser acessados e modificados por métodos da classe. Através dos atributos os objetos podem armazenar e representar informações relevantes para o problema que estão modelando, tornando a POO uma abordagem poderosa para organizar e manipular dados em programas de software (na figura 8 vimos que a classe Pessoa tem os atributos Nome e Idade).

Na linguagem C#, **campos** (ou *fields* em inglês) são variáveis declaradas diretamente dentro de uma classe, estrutura ou enumeração, e não são encapsuladas por métodos get ou set. Eles representam o estado interno ou os dados armazenados por um objeto ou instância. Ao contrário de propriedades, campos são tipicamente usados para armazenamento interno, enquanto propriedades expõem dados ao exterior da classe de maneira controlada.

Em outras palavras, o que até aqui chamamos de atributos, do ponto de vista da POO, equivale aos campos em C#. Tal distinção é importante pois, como veremos em breve, a palavra atributo tem outro significado na linguagem de programação. Do exemplo da figura 8 podemos então dizer, de modo preciso, que a classe Pessoa tem os campos Nome e Idade.

Em C# e na maioria das linguagens orientadas a objetos, os termos **variável de instância** e **campo** são usados de maneira intercambiável. Ambos se referem a variáveis declaradas dentro de uma classe (ou estrutura) mas fora de qualquer método, construtor ou propriedade, e representam o estado ou os dados associados a uma instância específica da classe. Quando você cria um objeto ou instância da classe, cada variável de instância ou campo terá um espaço de armazenamento distinto na memória para esse objeto. Essas variáveis ou campos são usados para armazenar o estado específico do objeto e diferem de variáveis estáticas, compartilhadas por todas as instâncias de uma classe.

Os campos geralmente têm modificadores de acesso – como private, public ou protected – que determinam a visibilidade e acessibilidade dessas variáveis dentro e fora da classe. Os campos são úteis para manter e gerenciar informações que fazem parte da instância de um objeto, permitindo que ele armazene dados de forma eficiente e isolada, seguindo os princípios de encapsulamento da POO.

Já as propriedades são membros especiais que fornecem um mecanismo flexível para ler, escrever ou calcular o valor de campos privados, permitindo um controle mais detalhado sobre o acesso e a modificação dos valores do que simples campos. Propriedades são definidas usando os métodos get e set, chamados **acessadores**; o acessador get retorna o valor da propriedade, enquanto o acessador set atribui um valor a ela.

Na figura 8 vimos que a classe Pessoa não tem propriedades, uma vez que não há acessadores; em contrapartida, na figura 9 vimos as propriedades Nome e Dominio na classe Deidade. Propriedades podem ser de leitura, escrita ou ambas, e através delas é possível implementar lógicas adicionais, como validações, ao obter ou definir valores, facilitando a manutenção e melhorando a segurança do código.

Já os atributos em C# são metadados que podem ser anexados a elementos do programa para fornecer informações adicionais; estas, por sua vez, podem ser usadas em tempo de execução ou design. Como vimos, não devem ser confundidos com atributos no contexto de características de uma classe, por exemplo, campos ou propriedades. Atributos em C# são classes que herdam da classe-base. Attribute e colocados acima de declarações usando colchetes (**[ ]**). Um exemplo comum é o [Obsolete], o qual indica que um componente é obsoleto. Atributos podem ter parâmetros e ser usados para influenciar o comportamento, a compilação ou fornecer insights sobre o código.

Constantes são variáveis cujos valores são definidos no momento da compilação e permanecem inalterados durante toda a execução do programa. São declaradas usando a palavra-chave **const** antes do tipo de dados; e uma vez atribuído um valor a uma constante, ele não pode ser modificado. Devido à sua natureza imutável, constantes devem ser inicializadas no momento da declaração. Elas são úteis quando temos valores que não devem ser alterados, como o valor de pi (constante matemática) ou o número de dias na semana. Ao usar constantes, o código se torna mais legível e menos propenso a erros, pois elimina números mágicos e valores arbitrários.

A figura 17 mostra um atributo (campo) privado do tipo string, chamado nomeSecreto, e uma propriedade pública do tipo string chamada Nome. Além disso, a classe definiu um campo público do tipo string chamado Origem, cujo valor "Monte Olimpo" é constante e, portanto, não pode ser alterado, já que foi declarado com a palavra-chave const.

```
1.  public class Semideus
2.  {
3.    private string nomeSecreto = "Herói Desconhecido";
4.    public string Nome { get; set; }
5.    public const string Origem = "Monte Olimpo";
6.  }
```

Figura 17 – Classe Semideus: exemplo de atributos, campos, propriedades e constantes

Note que o atributo nomeSecreto já começa com um valor predefinido "Herói Desconhecido". É possível criar um método construtor (público) que define esse valor, com o nome do semideus. Também podemos elaborar um novo método (abordaremos isso no tópico 2.3) para revelar o nome secreto (uma vez que ele é privado).

A figura 18 mostra um código-fonte com essas possibilidades.

```
1.  public class Semideus
2.  {
3.     private string nomeSecreto;
4.     public string Nome { get; set; }
5.     public const string ORIGEM = "Monte Olimpo";
6.
7.     public Semideus(string nome)
8.     {
9.        Nome = nome;
10.       nomeSecreto = "Herói Desconhecido";
11.    }
12.
13.    public void RevelarNomeSecreto()
14.    {
15.       Console.WriteLine($"O nome secreto é {nomeSecreto}.");
16.    }
17. }
```

Figura 18 – Classe Semideus: exemplo com método construtor

2.3 Métodos

Métodos em C# representam blocos de código que realizam uma tarefa específica e são uma das construções fundamentais na linguagem. Eles permitem que desenvolvedores organizem e reutilizem código, promovendo modularidade, legibilidade e manutenibilidade. Métodos são basicamente **procedimentos** ou **funções** associadas a uma classe ou objeto, têm um tipo de **retorno** – que pode ser qualquer tipo de dado ou void (quando o método não retorna nenhum valor) – e podem retornar qualquer tipo, incluindo tipos primitivos (int, double, char etc.), objetos, arrays, ou até mesmo outros métodos, desde que o tipo do método correspondente seja compatível.

O termo **return** tem significado especial e crucial. Se nos depararmos com a palavra-chave return dentro de um método, estaremos observando o mecanismo que permite a esse método enviar um valor de volta ao chamador (que invocou o método).

Quadro 2 – Tipos de retorno de método

Tipo de retorno	Descrição	Exemplo
void	O método não retorna valor	void PreparaRitual() {...}
int	O método retorna um valor inteiro	int ContarDeuses() {... return 12;}
string	O método retorna uma string	string NomeDeusPrincipal() {... return "Zeus";}
bool	O método retorna um valor booleano	bool DeusEstaPresente() {... return true;}

O quadro apresenta alguns tipos de retorno. A instrução return efetivamente encerra a execução do método e, se o método tiver um tipo de retorno diferente de void, a instrução return também enviará um valor de volta ao ponto onde o método foi chamado.

A definição do método dita se um valor de retorno é esperado ou não. Se o tipo de retorno for void, o método não retornará um valor; por outro lado, se houver um tipo de retorno especificado, o método é obrigado a retornar um valor desse tipo. A falha em retornar um valor resultará em erro de compilação. O termo **assinatura de um método** refere-se à combinação de seu nome, quantidade, ordem e tipo de parâmetro; é essencialmente uma forma de identificar o método de maneira única dentro de uma classe ou interface. Notavelmente, o tipo de retorno do método não faz parte da sua assinatura.

```
1.  class Olimpo
2.  {
3.    private int quantidadeDeuses = 12;
4.
5.    public void InvocarZeus()
6.    {
7.      Console.WriteLine("Zeus foi invocado!");
8.      // Este método é void, então não tem um 'return' com um valor.
9.    }
10.
11.   public int ContarDeusesNoOlimpo()
12.   {
13.     return quantidadeDeuses; // Retorna o número de deuses no Olimpo.
14.   }
15.
16.   public string DeusaDaSabedoria()
17.   {
18.     return "Atena"; // Retorna o nome da deusa da sabedoria.
19.   }
20.
21.   public bool EhZeus(string nome)
22.   {
23.     return nome == "Zeus"; // Retorna true se o nome for Zeus, caso
contrário, retorna false.
24.   }
25. }
```

Figura 19 – Retorno de métodos: diversos exemplos

Na figura temos quatro métodos (linhas 5, 11, 16 e 21). InvocarZeus não retorna nenhum valor e, portanto, não tem uma instrução return que retorna um valor. ContarDeusesNoOlimpo, por outro lado, retorna um valor inteiro representando o número de deuses no Olimpo. Da mesma forma, DeusaDaSabedoria retorna o nome da deusa da sabedoria como uma string. Finalmente, EhZeus é um método que aceita uma string como parâmetro e retorna um valor booleano, dependendo de a string ser "Zeus" ou não.

A instrução return também pode ser usada em métodos void para encerrar a execução prematuramente. Por exemplo, se um programador tentasse invocar o deus Apolo, mas para isso precisasse de um item especial, como uma lira, e ele não a tivesse, o método terminaria prematuramente usando a instrução return. Na figura 20 o método InvocarApolo verifica se temLira é true; se não for (ou seja, se ele não tem a lira), o programa imprimirá uma mensagem de aviso e usará a instrução return para terminar a execução do método sem tentar invocar Apolo. Se ele tiver a lira, a invocação prosseguirá normalmente.

```
1.  using System;
2.  public class Altar
3.  {
4.    private bool temLira = false;
5.    public void InvocarApolo()
6.    {
7.      // Verificando se temos a lira necessária para invocar Apolo.
8.      if (!temLira)
9.      {
10.         Console.WriteLine("Você não pode invocar Apolo sem uma
lira!");
11.         return; // Encerra a execução do método prematuramente.
12.     }
13.
14.     Console.WriteLine("Apolo foi invocado com sucesso!");
15.   }
16.   // Método para adicionar a lira ao altar, permitindo a invocação de
Apolo.
17.   public void AdicionarLira()
18.   {
19.     temLira = true;
20.     Console.WriteLine("Lira adicionada ao altar!");
21.   }
22.   public static void Main()
23.   {
24.     Altar altar = new Altar();
25.     altar.InvocarApolo(); // Tentará invocar, mas falhará pois não
temos a lira.
26.     altar.AdicionarLira(); // Adicionamos a lira ao altar.
27.     altar.InvocarApolo(); // Agora, a invocação será bem-sucedida.
28.   }
29. }
```

Figura 20 – Invocação de Apolo: uso do return no encerramento prematuro

Um método pode ter **parâmetros** que permitem a passagem de informações para ele, permitindo que os métodos sejam mais flexíveis e reutilizáveis ao operar em diferentes conjuntos de dados. A assinatura de um método – que inclui seu nome e seus tipos de parâmetro (mas não seus tipos de retorno) – deve ser única dentro de uma classe. Os parâmetros são definidos na assinatura e, ao chamar o método, os argumentos devem coincidir com os parâmetros.

```
1.  public class Criatura
2.  {
3.     public string Nome { get; set; }
4.     public string Descrever()
5.     {
6.        return $"A criatura é {Nome}.";
7.     }
8.  }
```

Figura 21 – Classe Criatura: exemplo de classe com método

A classe Criatura da figura tem um método chamado Descrever que, quando invocado, devolve uma string para quem o invocou com a descrição da criatura. Note que na declaração o tipo de retorno é string (linha 4), compatível com a cláusula de retorno (linha 6).

Métodos desempenham um papel crucial no conceito de encapsulamento em OOP: agir como interfaces para interagir com objetos e proteger os campos internos do objeto de acessos indesejados. Por exemplo, em vez de acessar diretamente um campo, você pode ter métodos get e set (ou propriedades em C#) que controlam o acesso e a modificação desse campo.

```
1.  public class Criatura
2.  {
3.     public string Nome { get; set; }
4.
5.     public Criatura(string nome)
6.     {
7.        Nome = nome;
8.     }
9.
10.    public string Descrever()
11.    {
12.       return $"A criatura é {Nome}.";
13.    }
14.
15.    public void Atacar()
16.    {
17.       Console.WriteLine($"{Nome} está atacando!");
18.    }
19. }
```

Figura 22 – Classe Criatura: inclusão de novos métodos

A classe Criatura foi expandida na figura e, além do método Descrever – que retorna uma descrição textual –, há o método Atacar, que simula a ação de ataque da criatura. Ambos os métodos utilizam a propriedade Nome para personalizar suas saídas. Observe também que foi inserido um método Construtor, que recebe um parâmetro do tipo string.

Existem dois mecanismos especiais chamados get e set, que são **acessadores** utilizados dentro de propriedades para controlar o acesso (leitura e escrita) a um campo particular de uma classe ou estrutura.

Eles fornecem uma maneira de encapsular a lógica associada ao acesso a um campo, permitindo maior controle sobre como os dados são manipulados e validados.

O get é utilizado para retornar o valor de um campo particular; se tentarmos acessar (ler) o valor de uma propriedade, a lógica dentro do bloco get será executada. Set é o acessador utilizado para definir (ou alterar) o valor de um campo particular; se tentarmos definir (escrever) o valor de uma propriedade, a lógica dentro do bloco set será executada. O valor que estamos tentando definir é acessível através da palavra-chave especial **value**. Por exemplo: imagine que estamos modelando um personagem da mitologia grega, como Hércules, e queremos definir e obter sua força.

```
1.  public class Hercules
2.  {
3.    private int forca;
4.
5.    public int Forca
6.    {
7.      get
8.      {
9.          // Aqui, simplesmente retornamos o valor da força.
10.         return forca;
11.     }
12.     set
13.     {
14.         // Suponha que a força de Hércules não possa ser negativa.
15.         if (value >= 0)
16.         {
17.           forca = value;
18.         }
19.         else
20.         {
21.           Console.WriteLine("A força não pode ser negativa!");
22.         }
23.     }
24.   }
25. }
```

Figura 23 – Classe Hercules: uso dos acessadores get e set

Na linha 5, a propriedade Forca encapsula o campo forca (linha 3). Se tentarmos obter o valor da propriedade Forca, o acessador get simplesmente retorna o valor atual de forca (linha 10); no entanto, ao tentar definir o valor da propriedade Forca, o acessador set garante que o valor não seja negativo (linha 15) antes de realmente alterar o campo forca (linha 17).

Propriedades com apenas o acessador get são consideradas somente leitura, e propriedades com apenas o acessador set são consideradas somente gravação. Se uma propriedade possuir ambos os acessadores, ela pode ser lida e modificada. A linguagem C# também introduziu propriedades autoimplementadas, que simplificam o código quando não é necessária lógica personalizada nos acessadores get e set. Nesse caso, não precisamos declarar um campo separado.

```
1.  public int Forca { get; set; }
```

Figura 24 – Forca: propriedade autoimplementada

Na criação da propriedade Forca, o compilador cria um campo backing (de suporte) não visível para armazenar o valor, e os acessadores get e set padrão são usados para acessar e modificar esse valor.

Por padrão, os parâmetros em C# são passados **por valor**. Isso significa que uma cópia do argumento é feita e passada para o método; dito de outro modo, modificações no parâmetro dentro do método não afetam o valor original fora dele. Todavia, ao usar a palavra-chave **ref** ou **out**, podemos passar um parâmetro **por referência**: o método opera na referência original do valor, e não em uma cópia. Portanto, quaisquer modificações no parâmetro dentro do método afetam o valor original fora dele.

Diferenças entre ref e out:

- **ref**: requer que a variável passada esteja inicializada antes de ser passada ao método.
- **out**: não exige que a variável passada esteja inicializada, mas requer que o método inicialize o parâmetro antes de terminar.

Há também uma categoria especial: os **métodos estáticos**, que são membros da classe, em vez de serem associados a uma instância específica dela. Não é necessário ter uma instância de um objeto para chamar um método estático; esses métodos são chamados no nível da classe, não no nível do objeto. São frequentemente usados para representar operações gerais para uma classe, em vez de operar no estado específico de uma instância.

Duas vantagens no uso de métodos estáticos: **eficiência** (como eles não operam em dados de instância, não é necessário criar um objeto para usá-los) e **conveniência** (eles podem ser chamados diretamente de uma classe, o que pode ser mais intuitivo em certos contextos). Mas também existem limitações: eles não podem acessar campos ou métodos de instância da classe e não podem ser sobrescritos em uma classe derivada; ou seja, a palavra-chave **virtual** não é aplicável a eles.

Lembrete

Já utilizamos o termo virtual anteriormente, na linha 10 da figura 5.

Para ilustrar melhor o método estático, considere uma classe chamada DeusesGregos. Ela tem um método estático que retorna o nome do deus ou deusa associado a um determinado conceito, conforme a linha 3 da figura 25.

```
1.  public class DeusesGregos
2.  {
3.     public static string DeidadeDoConceito(string conceito)
4.     {
5.        switch (conceito.ToLower())
6.        {
7.             case "sabedoria":
8.                return "Atena";
9.             case "mar":
10.               return "Poseidon";
11.            case "submundo":
12.               return "Hades";
13.            default:
14.               return "Desconhecido";
15.       }
16.    }
17. }
```

Figura 25 – Classe DeusesGregos: exemplo de métodos estáticos

Para chamar o método DeidadeDoConceito, não é necessário criar uma instância da classe DeusesGregos; em vez disso, chamamos o método diretamente na classe, conforme a figura 26. O mecanismo de acionamento do método se faz pelo operador ponto-final (.). Note que a linha 1 usa o nome da classe DeusesGregos, um símbolo de ponto-final e o nome do método DeidadeDoConceito, seguido do parâmetro "mar".

```
1.  string deusDoMar = DeusesGregos.DeidadeDoConceito("mar");
2.  Console.WriteLine(deusDoMar);  // Isso imprimirá "Poseidon"
```

Figura 26 – Chamada do método estático DeidadeDoConceito

O símbolo de ponto-final em C# é chamado de **operador de acesso** (*dot operator* em inglês) e é usado para acessar membros de um objeto ou tipo. Através dele podemos acessar campos, propriedades e métodos de uma instância de classe ou diretamente de um tipo (no caso de membros estáticos). O operador de acesso é o método-padrão e mais comum para acessar membros de um objeto em C#, mas não é a única maneira. Em contextos avançados, como a reflexão, podemos acessar e manipular membros de um objeto de maneiras diferentes. A reflexão permite obter informações sobre objetos, classes, interfaces etc. em tempo de execução, e executar operações com elas.

É possível também acessar membros através de ponteiros em contextos de código não seguro em C#, mas isso é menos comum e está além do escopo-padrão da linguagem. No código não seguro, o operador -> é usado em vez do operador de ponto-final.

```
1.  Deus zeus = new Deus();
2.  zeus.Nome = "Zeus";  // Acessando a propriedade Nome
3.  zeus.Falar();  // Acessando o método Falar
```

Figura 27 – Uso do operador de acesso (.)

A figura 27 demonstra o uso do operador ponto-final para acessar o conteúdo de propriedades (linha 2) e para acionar (ou chamar) o método (linha 3). Como veremos no tópico 2.5, a linha 1 cria um objeto da classe Deus.

```
1.  public class HeroisGregos
2.  {
3.    public static bool RealizouFeito(string heroi, string feito)
4.    {
5.      if (heroi == "Hércules" && feito == "12 trabalhos")
6.          return true;
7.      if (heroi == "Perseu" && feito == "Matar Medusa")
8.          return true;
9.      // ... outros heróis e feitos ...
10.
11.     return false;
12.   }
13. }
14.  // ... Main
15. bool herculesFeito = HeroisGregos.RealizouFeito("Hércules", "12
trabalhos");
16. Console.WriteLine(herculesFeito);  // Isso imprimirá "true"
```

Figura 28 – Classe HeroisGregos: outro exemplo do uso de métodos estáticos

Já na figura 28 temos a classe HeroisGregos, que contém um método estático para verificar se determinado herói realizou um feito específico. Esses exemplos ilustram como os métodos estáticos oferecem uma maneira de criar funcionalidades relevantes para uma classe como um todo, em vez de para instâncias específicas dessa classe. Em temas como mitologia grega – onde certos conceitos ou histórias são universais e imutáveis –, métodos estáticos são a maneira perfeita de representar essas ideias no código.

Por exemplo, o método "public static void Main(string[] args)", usado na linha 29 da figura 6, é especial na linguagem C# (e em muitas outras linguagens de programação) por ser o **ponto de entrada** de um programa. Se compilarmos e executarmos um programa C#, o ambiente de execução .NET (como o .NET Core ou .NET Framework) carrega o assembly (o arquivo compilado) e procura pelo método **Main** como ponto de entrada. Uma vez encontrado, ele começa a execução a partir desse método. Se tivermos vários métodos Main em diferentes classes, precisaremos especificar qual classe contém o ponto de entrada que se deseja usar durante a compilação.

A seguir, uma explicação desses componentes e de sua importância:

- **Main**: esse método (com M maiúsculo) é o ponto de partida de qualquer aplicativo C#. Quando um programa é iniciado, o sistema procura por esse método e começa a executá-lo. Se o método Main não estiver presente em um programa C#, não poderá ser executado.

- **public**: palavra-chave que indica o modificador de acesso do método. Torná-lo "public" significa que o método pode ser acessado por outras classes e métodos. No contexto de Main, geralmente

não é necessário ser public, pois o sistema operacional chama o Main diretamente, mas é uma convenção comum.

- **static**: essa palavra-chave indica que o método pertence à classe e não a uma instância específica dela. Isso é crucial para o método Main, porque o sistema precisa chamá-lo sem criar uma instância da classe.

- **void**: palavra-chave que indica que o método não retorna um valor. Alternativamente, o método Main também pode retornar um inteiro (int).

- **string[] args**: lista de argumentos que podem ser passados para o programa a partir da linha de comando. O sistema operacional repassa esses argumentos para o programa, e eles podem ser acessados e utilizados como qualquer outro array em C#. Alternativamente, também pode ser somente string[] ou somente void.

O método Main não precisa estar especificamente dentro de uma classe chamada Program. Esse nome para a classe que contém o método Main é uma convenção muitas vezes utilizada nos templates-padrão do Visual Studio para aplicativos de console, mas não é obrigatório.

Os chamados **métodos de extensão** permitem adicionar novos métodos a tipos existentes sem modificar o tipo original, e são úteis para adicionar funcionalidades a classes ou estruturas que não podemos ou não desejamos modificar. Para criar um método de extensão, o método deve ser estático e definido em uma classe estática. Além disso, o primeiro parâmetro do método deve usar a palavra-chave **this**, seguido pelo tipo que desejamos estender.

```
1.  public class Deus
2.  {
3.     public string Nome { get; set; }
4.  }
5.
6.  public static class ExtensaoDeus
7.  {
8.     public static bool EhReiDoOlimpo(this Deus deus)
9.     {
10.       return deus.Nome == "Zeus";
11.    }
12. }
```

Figura 29 – Classe estática ExtensaoDeus: exemplo de método de extensão

Na linha 6, vemos uma classe estática chamada ExtensaoDeus. Por enquanto não se preocupe com o fato da classe ser estática: vamos abordá-la em detalhes no tópico 2.7; nosso foco será o método estático EhReiDoOlimpo, da linha 8. Note que o tipo do primeiro parâmetro do método (Deus) é precedido pela palavra-chave **this**, e lembre-se que o objetivo de criar o método EhReiDoOlimpo era verificar se determinado deus é o rei do Olimpo, mas sem modificar a classe original (Deus).

A figura 30 ilustra um exemplo:

```
1.  Deus deus = new Deus { Nome = "Zeus" };
2.  bool reiDoOlimpo = deus.EhReiDoOlimpo();  // retornará true
```

Figura 30 – Método EhReiDoOlimpo: exemplo de um método de extensão

O C# também permite utilizar **parâmetros opcionais e nomeados**. Parâmetros opcionais permitem especificar um valor-padrão para um parâmetro – isso significa que, ao chamar o método, você pode omitir argumentos para quaisquer parâmetros opcionais.

```
1.  public class Heroi
2.  {
3.    public string Nome { get; set; }
4.
5.    public void RealizarMissao(string missao = "Missão desconhecida")
6.    {
7.      Console.WriteLine($"{Nome} está realizando: {missao}");
8.    }
9.  }
```

Figura 31 – Método RealizarMissao: exemplo de parâmetros opcionais

Na linha 5 o parâmetro do tipo string, "missao", foi inicializado com o valor-padrão "Missão desconhecida". Perceba que também é simples utilizar métodos com parâmetros opcionais: a figura 32 aponta dois exemplos – acionamento do método sem parâmetro algum (e nesse caso prevalece o valor-padrão) e a chamada do método com o parâmetro "12 camadas".

```
1.  Heroi heroi = new Heroi { Nome = "Hércules" };
2.  heroi.RealizarMissao();  // imprimirá "Hércules está realizando:
Missão desconhecida"
3.  heroi.RealizarMissao("12 trabalhos");  // imprimirá "Hércules está
realizando: 12 trabalhos"
```

Figura 32 – Utilização do método RealizarMissao

Agora imagine o que aconteceria se tivéssemos um método com três parâmetros opcionais e somente quiséssemos passar um deles – por exemplo o terceiro da lista – mantendo os dois primeiros com os valores-padrão do método. A fim de facilitar a tarefa, na chamada de um método é possível usar argumentos nomeados para definir valores para parâmetros específicos com base em seus nomes, independentemente de sua posição na lista de parâmetros.

A figura 33 ilustra essa possibilidade. Na declaração do método, o parâmetro "nome" não é opcional.

```
1.  public void RegistrarDeus(string nome, string dominio =
"Desconhecido", bool ehOlimpiano = true)
2.  {
3.     // Lógica do método
4.  }
```

Figura 33 – Método RegistrarDeus: exemplo de parâmetros opcionais mesclado com parâmetros não opcionais

A utilização de parâmetros nomeados no código é simples; basta mencionar o nome do parâmetro, independente de sua ordem na lista – a figura 34 ilustra esse caso. Note que a ordem em que os parâmetros são fornecidos não coincide com a ordem na assinatura do método. Isso é útil especialmente se um método tiver vários parâmetros opcionais e quisermos especificar um valor para apenas um deles, ignorando os outros.

```
1.  RegistrarDeus(dominio: "Mar", nome: "Poseidon", ehOlimpiano: true);
```

Figura 34 – Uso do método RegistrarDeus

Perceba que em alguns códigos-fonte deste livro-texto (por exemplo na linha 7 da figura 31) há a instrução "Console.WriteLine(...". Agora fica mais fácil perceber que essa expressão é a invocação do método WriteLine de uma classe chamada Console. Em outras palavras, é um método da classe Console para escrever uma linha de texto na saída-padrão, normalmente o console ou terminal.

Para exibir informações ao usuário, uma das funções mais básicas e usadas em programas C# é passar o texto a ser escrito como parâmetro para o método. A classe Console não foi criada para este livro-texto; é uma parte do .NET Framework, desenvolvido pelos programadores da empresa Microsoft em 2000. Portanto, ao invocá-lo nos exemplos, estamos reutilizando o código.

```
1.  // Exemplo básico:
2.  Console.WriteLine("Olá, mundo!");
3.
4.  // Usando formatação:
5.  string deus = "Zeus";
6.  Console.WriteLine("O deus supremo é {0}.", deus);
7.
8.  // Usando interpolação de string (introduzida em C# 6):
9.  Console.WriteLine($"O deus supremo é {deus}.");
10.
11. // Escrevendo valores numéricos:
12. int idadeDeZeus = 3000;
13. Console.WriteLine($"Zeus tem {idadeDeZeus} anos.");
```

Figura 35 – Método Writeline: exemplos de uso

A figura apresenta quatro possibilidades de invocar o método, que é frequentemente usado em aplicações de console para fornecer feedback ao usuário, mostrar os resultados de cálculos ou exibir mensagens de erro. Por ser fundamental, é geralmente uma das primeiras funções que os novatos em C# aprendem ao começar a programar. O método Console.WriteLine pode ter possibilidades distintas porque é **sobrecarregado** para aceitar diferentes tipos e quantidades de argumento. Isso permite que ele imprima vários tipos de dados sem precisar converter tudo para uma string primeiro.

```
1.  public class Divindade
2.  {
3.    public void Saudacao()
4.    {
5.      Console.WriteLine("Olá, divindade!");
6.    }
7.
8.    public void Saudacao(string nome)
9.    {
10.     Console.WriteLine($"Olá, {nome}!");
11.   }
12.
13.   public void Saudacao(string nome, string titulo)
14.   {
15.     Console.WriteLine($"Olá, {titulo} {nome}!");
16.   }
17. }
18.
19. class Program
20. {
21.   static void Main()
22.   {
23.     Divindade divindade = new Divindade();
24.
25.     divindade.Saudacao();                  // Olá, divindade!
26.     divindade.Saudacao("Zeus");            // Olá, Zeus!
27.     divindade.Saudacao("Zeus", "Senhor"); // Olá, Senhor Zeus!
28.   }
29. }
```

Figura 36 – Método Saudacao: exemplo de sobrecarga de método

O método Saudacao foi sobrecarregado três vezes: uma versão sem parâmetros, uma com um parâmetro de tipo string e uma com dois parâmetros de tipo string. As linhas 25 a 27 mostram as invocações do método (dentro do Main), e a sobrecarga garante que o compilador determine qual versão do método Saudacao chamar com base nos argumentos fornecidos nas invocações.

Sobrecarga é um conceito que permite múltiplos métodos com o mesmo nome na mesma classe, mas com diferentes listas de parâmetros. A sobrecarga de método é feita dentro da mesma classe (ou em classes derivadas, se herança estiver envolvida). A distinção entre diferentes sobrecargas do mesmo método é baseada nos tipos, na ordem e no número de parâmetros. O compilador usa essas diferenças para determinar qual versão do método o programador está tentando invocar.

Além disso, a sobrecarga não é determinada pelo tipo de retorno, portanto não podemos criar uma sobrecarga de método apenas mudando o tipo de retorno. A sobrecarga pode ser aplicada a métodos, construtores e operadores, mas não a propriedades ou campos. Em resumo, sobrecarga de método é uma maneira de fornecer flexibilidade e clareza ao definir funcionalidades semelhantes, permitindo que o programador use o mesmo nome de método em diferentes contextos, mas com diferentes parâmetros.

Os métodos desempenham um papel fundamental na estrutura das classes em C# e em muitas outras linguagens de POO: encapsulam comportamentos, promovem reutilização de código, simplificam a manutenção e fornecem uma interface clara para interagir com objetos, tornando-os essenciais para desenvolver um software eficaz e de alta qualidade.

2.4 Construtores e desconstrutores

Construtor é um método especial de uma classe ou estrutura em C# automaticamente invocado quando um objeto daquela classe é criado. Seu principal objetivo é inicializar um objeto, e suas duas características básicas são: ter o mesmo nome da classe e não ter valor de retorno, nem mesmo void.

Toda classe em C# tem um construtor, mesmo que não o declaremos explicitamente. Se você não fornecer um construtor para sua classe, o C# criará automaticamente um construtor-padrão sem parâmetros, que será chamado de **construtor padrão implícito**. Ele não faz nada além de criar o objeto; também não inicializa nenhum membro de dados, a menos que esses membros tenham inicializadores de campo. Assim, os tipos de valor são inicializados com seus valores-padrão (por exemplo, números inteiros são inicializados como 0), e os tipos de referência são inicializados como null.

```
1.  public class Deus
2.  {
3.    public string Nome;
4.    public string Dominio;
5.  }
6.
7.  class Program
8.  {
9.    static void Main()
10.   {
11.     Deus apolo = new Deus();
12.     Console.WriteLine(apolo.Nome);  // Saída: (uma linha em branco,
pois a string é null por padrão)
13.   }
14. }
```

Figura 37 – Classe Deus: exemplo de método construtor padrão implícito

A classe Deus tem dois campos (Nome e Dominio), mas não tem métodos explicitamente declarados. A linha 11 cria um objeto da classe Deus (analisaremos objetos em detalhes no tópico 2.5). Uma vez que não há método construtor explícito, o valor do campo Nome do deus criado é preenchido com null. Mas é importante notar que, se o programador declarar qualquer construtor para sua classe (seja ele

parametrizado ou não), o C# não fornecerá um construtor padrão implícito. Se o programador ainda desejar um construtor sem parâmetros nesse cenário, terá que declará-lo explicitamente.

É possível ter mais de um construtor por classe, o que permite inicializar o objeto de maneiras diferentes, conforme a necessidade capturada na modelagem a partir da extração dos requisitos do sistema.

Lembrete

Estudamos a sobrecarga de métodos no tópico 2.3.

```
1.  public class Deus
2.  {
3.    public string Nome { get; set; }
4.    public string Dominio { get; set; }
5.
6.    // Construtor padrão
7.    public Deus()
8.    {
9.      Nome = "Desconhecido";
10.     Dominio = "Desconhecido";
11.   }
12.
13.   // Construtor com parâmetros
14.   public Deus(string nome, string dominio)
15.   {
16.     Nome = nome;
17.     Dominio = dominio;
18.   }
19. }
```

Figura 38 – Classe Deus: métodos construtores distintos

No exemplo temos dois construtores – um padrão (sem parâmetros) e um com parâmetros (linhas 7 e 14, respectivamente). Eles ajudam a inicializar o objeto Deus de maneiras diferentes, conforme a figura 39:

```
1.  Deus zeus = new Deus();   // Usa o construtor padrão. Nome e Dominio
serão "Desconhecido"
2.
3.  Deus poseidon = new Deus("Poseidon", "Mar");   // Usa o construtor
com parâmetros. Nome será "Poseidon" e Dominio será "Mar"
```

Figura 39 – Utilização dos distintos construtores da classe Deus

Em C# o termo **desconstrutor** se relaciona ao recurso introduzido na versão 7, chamado desconstrução, que permite decompor objetos em variáveis individuais – isso é particularmente útil com tipos de valor como **tuplas**, um conjunto ordenado de elementos (falaremos delas em detalhes no tópico 5.3); é uma estrutura de dados que permite armazenar múltiplos valores, possivelmente de tipos diferentes, como uma única unidade.

A desconstrução permite extrair múltiplos campos, propriedades ou outros valores de um objeto em muitas variáveis. Um objeto que suporta desconstrução deve ter um método Deconstruct que o desmonte. Além disso, esse método não retorna valores, mas usa parâmetros out para isso.

```
1.  public class Deus
2.  {
3.     public string Nome { get; }
4.     public string Dominio { get; }
5.
6.     public Deus(string nome, string dominio)
7.     {
8.        Nome = nome;
9.        Dominio = dominio;
10.    }
11.
12.    // Método de desconstrução
13.    public void Deconstruct(out string nome, out string dominio)
14.    {
15.       nome = Nome;
16.       dominio = Dominio;
17.    }
18. }
```

Figura 40 – Método Deconstruct da classe Deus

O método Deconstruct na figura não retorna um valor; em vez disso, utiliza parâmetros out para retornar múltiplos valores. Já na figura 41 será criada uma instância da classe Deus, e a desconstrução será utilizada para extrair seus valores.

```
1.  var zeus = new Deus("Zeus", "Céu");
2.
3.  // Usando a desconstrução para extrair valores
4.  (string deusNome, string deusDominio) = zeus;
5.  Console.WriteLine($"Nome: {deusNome}, Dominio: {deusDominio}");
```

Figura 41 – Método Deconstruct

Não é necessário ter um método construtor explícito para ter um desconstrutor: eles são independentes. A figura 42 ilustra a criação de uma classe sem construtor explícito; na linha 6, "nome" e "poder" recebem os valores "Nome" e "Poder" da propriedade. Já a figura 43 demonstra sua utilização.

```
1.  public class Deus
2.  {
3.    public string Nome { get; set; }
4.    public string Poder { get; set; }
5.
6.    public void Deconstruct(out string nome, out string poder)
7.    {
8.      nome = Nome;
9.      poder = Poder;
10.   }
11. }
```

Figura 42 – Método Deconstruct sem construtor explícito

```
1.  Deus zeus = new Deus { Nome = "Zeus", Poder = "Relâmpagos" };
2.  (string deusNome, string deusPoder) = zeus; // Desconstrução
```

Figura 43 – Utilização do método Deconstruct sem construtor explícito

Há outro conceito em C# cujo nome se assemelha aos descontrutores: os destrutores. Apesar da semelhança da grafia ou sonoridade, são conceitos completamente distintos. Um **destrutor** libera recursos ou realiza outras operações de limpeza antes que um objeto seja recolhido pelo coletor de lixo (*garbage collector*), característica fundamental da plataforma .NET, da qual C# é uma das principais linguagens. Ele gerencia a alocação e a liberação automática da memória para as aplicações – processo crucial para garantir a eficiência e estabilidade das aplicações. No C#, destrutores somente podem ser usados em classes e têm o mesmo nome da classe, precedido pelo símbolo **~**. Eles não podem ser herdados nem sobrecarregados, e não têm parâmetros nem modificadores de acesso.

```
1.  public class Deus
2.  {
3.    public string Nome { get; set; }
4.    public string Dominio { get; set; }
5.
6.    // ... construtores ...
7.
8.    // Destrutor
9.    ~Deus()
10.   {
11.     Console.WriteLine($"{Nome} foi enviado para o Tártaro.");
12.   }
13. }
```

Figura 44 – Classe Deus: definição de um destrutor

O destrutor da linha 9 imprimirá uma mensagem quando o objeto Deus for finalizado pelo coletor de lixo; ou seja, quem invoca o destrutor é o coletor. Seguindo nossa analogia mitológica, construtores são como o nascimento de um deus, trazendo-o à existência com certos atributos e poderes. Destrutores, por outro lado, são como o fim simbólico dessa divindade, garantindo que todos os rastros de sua influência sejam removidos do mundo mortal.

É importante notar que, em C#, um programador raramente precisará escrever um destrutor explicitamente, uma vez que o coletor de lixo lida com a maioria das tarefas de limpeza automaticamente. Destrutores geralmente são necessários apenas se uma classe estiver gerenciando recursos externos diretamente, como janelas de sistema, conexões de rede ou arquivos.

2.5 Criação de objetos

A criação de objetos é peça fundamental na POO. A capacidade de criar e manipular objetos é o que dá vida às aplicações, permitindo que os desenvolvedores representem entidades, processos ou conceitos do mundo real dentro de um ambiente de software. Como vimos, uma classe pode ser pensada como modelo para criar objetos, pois define atributos (campos ou propriedades) e comportamentos (métodos) de um objeto. Em resumo, criar um objeto significa criar uma instância dessa classe, um processo conhecido como **instanciação**, em que se usa a palavra-chave new seguida pelo nome da classe e, se necessário, quaisquer argumentos que o construtor dessa classe possa exigir. Ao se criar a instância de uma classe, reserva-se espaço na memória para essa instância, iniciada com quaisquer valores-padrão ou valores fornecidos.

Ao criar um objeto, não estamos apenas produzindo a cópia passiva de um modelo, mas trazendo à vida uma entidade com estado e comportamento. Esse objeto pode interagir com outros objetos, pode ser modificado e pode desencadear uma variedade de ações e comportamentos dentro do software. Outra consideração importante é o escopo e a vida útil de um objeto. Em C#, um objeto permanecerá na memória enquanto houver referência a ele; se não houver mais referências apontando para esse objeto, ele se torna elegível para coleta de lixo (*garbage collection*), ou seja, o gerenciador de memória do .NET Framework eventualmente limpará esse espaço de memória, garantindo que os recursos do sistema sejam utilizados de maneira eficiente.

Convém lembrar também que os vários modificadores de acesso (como public, private e protected) permitem aos desenvolvedores controlar como e onde seus objetos são acessados e modificados. Isso não apenas protege a integridade do objeto, mas também ajuda a manter o código organizado e previsível. Além disso, quando criamos um objeto, o que realmente temos em mãos é uma referência para esse objeto, e não o objeto em si – referência que aponta para a localização do objeto na memória. Pode-se pensar nisso como um endereço que indica onde o objeto reside no espaço de memória do sistema. Quando atribuímos um objeto a outro, estamos realmente passando a referência, e não uma cópia do objeto. Isso significa que ambas as variáveis agora apontam para o mesmo local na memória e, portanto, para o mesmo objeto.

Teoricamente, não há limite fixo para o número de objetos que podemos criar em C#, mas na prática o número de objetos que podem ser criados é limitado pela quantidade de memória disponível no sistema. Cada objeto criado consome certa quantidade de memória (que depende dos campos e métodos da classe), e quando a memória se esgota não é possível criar mais objetos. Em sistemas modernos, com grandes quantidades de *random access memory* (RAM), geralmente é possível criar milhões de objetos simples sem encontrar problemas. Contudo, deve-se sempre estar ciente do consumo de memória, especialmente em aplicações que requerem alta performance ou que serão executadas em dispositivos com recursos limitados.

As linhas entre 2 e 17 da figura 45 representam a criação da classe Deus. A classe não é o objeto e, portanto, até a linha 17 ele não existe. Da linha 19 à 31 há uma nova classe chamada MeusPrimeirosObjetos, que contém um método estático chamado Main, o ponto inicial do programa (como vimos no tópico 2.3). Na linha 23 temos a criação efetiva do objeto, a instanciação propriamente dita, e a partir dessa linha o objeto, cujo nome da referência é zeus, passa a residir na memória do computador.

Note que zeus é o nome do objeto; a classe à qual ele se refere é Deus. Observe também que a palavra-chave new foi usada para criar o objeto, bem como o método construtor da classe (que recebeu como parâmetros o Nome e o Dominio do novo Deus).

```
1.  using System;
2.  public class Deus
3.  {
4.    public string Nome { get; set; }
5.    public string Dominio { get; set; }
6.
7.    public Deus(string nome, string dominio)
8.    {
9.      Nome = nome;
10.     Dominio = dominio;
11.   }
12.
13.   public void ExibirPoderes()
14.   {
15.     Console.WriteLine($"{Nome} eh o(a) deus(a) do(a) {Dominio}.");
16.   }
17. }
18.
19. public class MeusPrimeirosObjetos
20. {
21.   public static void Main(string[] args)
22.   {
23.     Deus zeus = new Deus("Zeus", "Ceu e Trovao");
24.     Console.WriteLine(zeus.Nome); // Saída: Zeus
25.     zeus.ExibirPoderes(); // Saída: Zeus é o(a) deus(a) do(a) Céu e
Trovão.
26.     Deus atena = new Deus("Atena", "Sabedoria");
27.     Deus afrodite = new Deus("Afrodite", "Amor");
28.     Deus hades = new Deus("Hades", "Submundo");
29.     atena.ExibirPoderes(); // Saída: Atena eh o(a) deus(a) do(a)
Sabedoria.
30.   }
31. }
```

Figura 45 – Classe Deus: exemplo para instanciar objeto

Na linha 24 da figura 45 há uma mensagem na tela sobre o acesso à propriedade Nome do objeto (zeus.Nome). Na linha 25 o método ExibirPoderes de zeus é invocado (zeus.ExibirPoderes()). Nas linhas seguintes, outros três objetos da classe Deus foram instanciados: atenas, afrodites e hades, com suas propriedades próprias.

Em resumo, a mitologia grega, com seu panteão de deuses e suas complexas relações, pode ser mapeada elegantemente em C# através de classes e objetos. Cada deus pode ser representado como objeto, com propriedades que definem seus atributos e métodos que determinam suas ações. Ao criar e interagir com esses objetos, podemos modelar e simular o mundo da mitologia grega, demonstrando o poder e a flexibilidade da POO em C#. Assim como os antigos gregos encontravam significado em suas histórias, os programadores encontram eficiência e clareza ao criar e manipular objetos em suas aplicações.

2.6 Referência this

Em métodos de instância ou propriedades de uma classe, **this** é usado para referir-se à instância atual da classe em que o método ou a propriedade está sendo executada. Pode ser útil para desambiguar membros da classe e parâmetros, especialmente quando têm o mesmo nome.

```
1.  public class Heroi
2.  {
3.    private string nomeDeus;
4.
5.    public Heroi(string nomeDeus)
6.    {
7.      this.nomeDeus = nomeDeus;
8.    }
9.  }
```

Figura 46 – Uso de this como referência à instância atual

Na figura 46 a classe Heroi tem um campo chamado nomeDeus (linha 3), e o parâmetro do método construtor também é chamado nomeDeus (linha 5), gerando ambiguidade. Na linha 7, this.nomeDeus refere-se ao parâmetro do método construtor, e nomeDeus ao campo da classe.

```
1.  public class Deidade
2.  {
3.    private string nome;
4.    private string dominio;
5.
6.    public Deidade(string nome) : this(nome, "Desconhecido") { }
7.
8.    public Deidade(string nome, string dominio)
9.    {
10.     this.nome = nome;
11.     this.domínio = dominio;
12.   }
13. }
```

Figura 47 – Uso de this como chamada para outro construtor na mesma classe

A palavra-chave this também pode ser usada dentro de um construtor para invocar outro construtor na mesma classe. Isso é útil se a classe tiver múltiplos construtores e quisermos evitar a repetição de código. Na figura 47 existem 2 métodos construtores, e na linha 6 o primeiro construtor (que somente tem um

parâmetro, chamado nome) está invocando o segundo construtor (que está na linha 8). Na invocação se usa o this em vez do nome do construtor, evitando a repetição da palavra Deidade e facilitando o entendimento.

Em métodos de extensão, o primeiro parâmetro especifica a classe que está sendo estendida e é precedido pela palavra-chave this, indicando que o método pode ser chamado como se fosse um método de instância da classe. Na figura 48, com a palavra-chave this no parâmetro, o método de extensão RevelarDestino atua como um poder inerente da classe Mortal.

```
1.  public static class ExtensoesDivinas
2.  {
3.     public static void RevelarDestino(this Mortal mortal)
4.     {
5.        // Descubra o destino do mortal aqui.
6.     }
7.  }
```

Figura 48 – Uso de this como método de extensão

2.7 Classes estáticas

Uma classe estática é definida usando a palavra-chave static, e sua principal característica é não ser instanciada; em outras palavras, não podemos criar um objeto de classe estática usando o operador new. Uma vez que não se pode criar instâncias de classes estáticas, todos os seus membros (campos, propriedades e métodos) devem ser obrigatoriamente estáticos. Membros estáticos pertencem à classe em si e não a uma instância da classe. Para acessar um membro estático, usa-se o nome da classe, seguido pelo nome do membro.

No exemplo da figura 49, o método Profecia pode ser chamado diretamente no nome da classe como OraculoDeDelfos.Profecia, sem a necessidade de criar uma instância do oráculo.

```
1.  public static class OraculoDeDelfos
2.  {
3.     public static string Profecia()
4.     {
5.        return "Um grande herói surgirá!";
6.     }
7.  }
```

Figura 49 – Classe OraculodeDelfos: exemplo de classe estática

Apesar de uma classe estática não ter construtores de instância, ela pode ter um construtor estático. Um construtor estático é chamado automaticamente quando a classe é carregada pela primeira vez, antes de qualquer membro estático ser acessado ou referenciado. Esse construtor não pode ser chamado diretamente e somente será executado uma vez durante a execução do programa.

O construtor estático geralmente é usado para inicializar qualquer campo estático ou executar ações que precisam ser realizadas antes de a classe ser usada.

```
1.  public static class CronicaDosDeuses
2.  {
3.     public static void NarrarHistoriaDeZeus()
4.     {
5.        Console.WriteLine("Zeus, o rei dos deuses...");
6.     }
7.
8.     public static void NarrarHistoriaDeAthena()
9.     {
10.       Console.WriteLine("Athena, deusa da sabedoria...");
11.    }
12. }
```

Figura 50 – Classe CronicaDosDeuses: exemplo de utilização de classe estática

Classes estáticas são frequentemente usadas como utilitários ou ferramentas. Por exemplo, classes que fornecem funções matemáticas, operações de manipulação de strings ou funcionalidades específicas que não necessitam manter um estado entre chamadas geralmente são projetadas como classes estáticas. Na figura 50, por exemplo, há uma classe que reúne histórias dos deuses, e a classe CronicaDosDeuses serve como repositório de narrativas que podem ser invocadas sem a necessidade de criar uma instância.

As vantagens em usar classes estáticas são desempenho e organização do código. Uma vez que não é necessário instanciar, o acesso a membros de uma classe estática pode ser ligeiramente mais rápido. Além disso, classes estáticas são ideais para agrupar métodos relacionados que não precisam acessar nem modificar o estado do objeto.

Uma das limitações de classes estáticas é que elas não podem ser herdadas por outras classes. Essa limitação é lógica, porque os membros estáticos pertencem à classe e não a instâncias, tornando a herança irrelevante nesse contexto, além de não poderem implementar interfaces. Uma vez que a implementação de uma interface exige a instanciação da classe para acessar membros implementados, isso não se alinha com a natureza de uma classe estática.

Há várias classes estáticas populares e amplamente utilizadas em C#. Uma das mais conhecidas é a System.Math, que fornece funções matemáticas básicas, e todos os seus membros são estáticos. Ou seja, não podemos (e não precisamos) criar uma instância da classe Math para usar suas funções; em vez disso, chamamos as funções diretamente através do nome da classe.

A figura 51 mostra o uso da classe Math sem a necessidade de criar objetos da classe.

```
1.  double raiz = Math.Sqrt(16);      // Calcula a raiz quadrada
2.  double potencia = Math.Pow(2, 3);// Eleva 2 à potência de 3
3.  double maior = Math.Max(5, 7);    // Retorna o maior valor entre 5 e 7
```

Figura 51 – Classe Math: exemplo de classe estática muito utilizada

Além da classe Math, outras classes estáticas notáveis em C# incluem:

- **System.Console**: embora não seja completamente estática (pois tem um construtor protegido), é predominantemente usada por seus membros estáticos para manipular a entrada e saída do console.

- **System.Environment**: fornece informações sobre o ambiente de execução atual e várias utilidades relacionadas ao sistema.

Estas são apenas algumas das classes estáticas populares em C#, projetadas dessa forma para oferecer funcionalidades específicas que não necessitam manter estado entre chamadas e são mais convenientemente acessadas diretamente através da classe.

2.8 Interfaces e classes abstratas

No C# as duas principais maneiras de criar abstrações são através de interfaces e classes abstratas. Embora ambos os conceitos tenham semelhanças, servem a propósitos diferentes e têm características distintas.

Interface é um contrato que outras classes podem implementar. Uma interface define um conjunto de assinaturas de métodos, propriedades, eventos ou indexadores sem implementar a funcionalidade real desses membros. Em outras palavras, uma interface define o "o quê", mas não o "como". Uma de suas características importantes é permitir a implementação de múltiplas heranças. Em C#, uma classe somente pode herdar de uma classe-base, mas pode implementar múltiplas interfaces.

```
1.  public interface IVoador
2.  {
3.     void Voar();
4.  }
5.
6.  public interface IAtacante
7.  {
8.     void Atacar();
9.  }
```

Figura 52 – Interfaces IVoador e IAtacante

Suponhamos diferentes entidades mitológicas que podem voar e atacar. A figura 52 apresenta duas interfaces que definem essas possibilidades, sem detalhar como serão implementadas; já na figura 53 a classe Pegasus, um cavalo alado, pode implementar ambas as interfaces a seu modo.

```
1.  public class Pegasus : IVoador, IAtacante
2.  {
3.     public void Voar()
4.     {
5.        Console.WriteLine("Pegasus está voando.");
6.     }
7.
8.     public void Atacar()
9.     {
10.       Console.WriteLine("Pegasus ataca com seus cascos.");
11.    }
12. }
```

Figura 53 – Classe Pegasus: exemplo de implementação de interfaces

Harpias são criaturas frequentemente descritas na mitologia grega como pássaros com rosto de mulher, conhecidas por serem rápidas, agressivas e voarem alto. Na figura 54, a Harpia implementa as interfaces IVoador e IAtacante (por convenção, as interfaces começam com I). Assim como o Pegasus, ela pode voar e atacar, mas de maneira distinta. Através das interfaces, temos flexibilidade para definir ações a cada criatura, mantendo uma estrutura uniforme sobre as capacidades que cada uma deve ter.

```
1.  public class Harpia : IVoador, IAtacante
2.  {
3.     public void Voar()
4.     {
5.        Console.WriteLine("A Harpia alça voo com suas poderosas asas.");
6.     }
7.
8.     public void Atacar()
9.     {
10.       Console.WriteLine("A Harpia ataca com suas garras afiadas e seu
bico.");
11.    }
12. }
```

Figura 54 – Classe Harpia: outro exemplo de implementação de interfaces

Como vimos, uma interface é declarada usando a palavra-chave interface, e seus membros não podem ter modificadores de acesso; são sempre public e não podem ser private nem protected. Uma classe ou estrutura que implementa uma interface deve fornecer implementação para todos os membros. Além disso, como nos exemplos Pegasus e Harpia, uma classe pode implementar várias interfaces, o que oferece uma forma de herança múltipla, já que uma classe pode herdar de apenas uma classe-base. Por fim, interfaces podem herdar de outras interfaces, criando uma cadeia de herança entre elas, no entanto uma interface não pode herdar de uma classe ou estrutura.

Interfaces ajudam a separar definição e implementação, permitindo que o programador defina operações sem especificar como são realizadas. Permite também que objetos de diferentes tipos sejam tratados como objetos de um tipo comum. Além disso, ao adicionar novas interfaces (em vez de alterar as existentes), os sistemas podem ser facilmente estendidos sem quebrar o código existente. Uma vez

que as classes podem implementar várias interfaces, conseguem ter múltiplas capacidades associadas a cada interface implementada.

Elas são fundamentais para uma programação eficaz e reutilizável, promovem a separação clara entre definição e implementação, e oferecem um mecanismo robusto para criar sistemas extensíveis e flexíveis. Ao trabalhar com grandes sistemas ou bibliotecas, o uso adequado das interfaces pode ser a diferença entre um design de software bem-sucedido e um problemático.

Uma **classe abstrata** é semelhante a uma interface, mas permite fornecer alguma implementação-padrão para seus membros. Essa classe pode ter métodos com implementação completa, mas não pode ser instanciada diretamente; em vez disso, outras devem herdar dessa classe abstrata e implementar métodos abstratos sem implementação. A principal diferença entre classes abstratas e interfaces é que uma classe pode herdar apenas de uma classe abstrata (devido à restrição de herança única em C#), mas pode implementar várias interfaces.

Vamos considerar que todos os deuses do Olimpo têm poderes e um nome, mas cada um manifesta seu poder de maneira única. A figura 55 representa essa ideia:

```
1.  public abstract class Deus
2.  {
3.     public string Nome { get; set; }
4.
5.     public Deus(string nome)
6.     {
7.        Nome = nome;
8.     }
9.
10.    public abstract void ManifestarPoder();
11. }
```

Figura 55 – Classe Deus: exemplo de classe abstrata

Podemos fazer com que Zeus, o rei dos deuses, herde da classe Deus. A figura 56 ilustra esse cenário.

```
1.  public class Zeus : Deus
2.  {
3.     public Zeus() : base("Zeus") { }
4.
5.     public override void ManifestarPoder()
6.     {
7.        Console.WriteLine("Zeus lança um raio!");
8.     }
9.  }
```

Figura 56 – Classe Zeus: herança da classe abstrata Deus

Uma classe abstrata serve de base para outras classes. Embora seja semelhante a uma interface, ela difere principalmente porque pode implementar métodos, propriedades, indexadores e eventos por padrão. A característica principal de uma classe abstrata é que não podemos criar uma instância direta dela; em vez disso, devemos herdar dela.

Conforme o exemplo, uma classe abstrata é declarada usando a palavra-chave abstract, e não podemos criar uma instância de uma classe abstrata. O principal propósito de uma classe abstrata é fornecer uma base comum para outras classes, e os membros das classes abstratas podem ter modificadores de acesso – como public, private e protected. Ao contrário das interfaces, classes abstratas podem conter campos (variáveis de instância), permitindo que elas mantenham estados.

Classes abstratas desempenham papel fundamental na arquitetura e no design orientado a objetos, pois permitem definir estruturas e comportamentos que outras classes podem herdar e expandir, sem comprometer a integridade ou a intenção original da classe-base. Ao entender quando e como usar classes abstratas (em contraste com interfaces e classes concretas), os desenvolvedores podem criar sistemas mais flexíveis, robustos e bem organizados.

Tanto interfaces quanto classes abstratas são ferramentas poderosas em C# que permitem aos desenvolvedores criar abstrações flexíveis e reutilizáveis (o quadro 3 compara esses dois mecanismos). Interfaces fornecem uma maneira de definir contratos que as classes podem implementar, já classes abstratas oferecem uma maneira de fornecer alguma implementação-base enquanto ainda definem uma estrutura que as classes derivadas devem seguir. A escolha adequada geralmente depende das necessidades específicas do design do software.

Quadro 3 – Comparativo entre interface e classe abstrata

Critério	Interface	Classe abstrata
Definição	Contrato que define a assinatura de métodos, propriedades e outros membros sem implementá-los	Pode ter tanto membros abstratos (sem implementação) quanto concretos (com implementação)
Instanciação	Não é possível instanciar diretamente	Não é possível instanciar diretamente
Herança	Uma classe pode implementar várias interfaces	Uma classe somente pode herdar de uma classe abstrata
Membros	Somente podem conter declarações	Podem ter declarações e implementações

2.9 Hierarquia de classes

Hierarquia de classes em C# é uma estrutura crucial que ajuda os desenvolvedores a organizar e estruturar seu código de maneira lógica e intuitiva. Refere-se à maneira como as classes são organizadas uma em relação às outras, normalmente para representar algum tipo de relação "é um tipo de". Pela hierarquia é possível identificar classes mais gerais e mais específicas, permitindo reutilizar códigos e facilitando seu entendimento e manutenção.

No centro da hierarquia de classes em C#, há as classes-base e as classes derivadas. Uma classe-base pode servir como ponto de partida para outras classes; estas, por sua vez, que derivam da classe-base, são chamadas classes derivadas.

Em C# e no .NET Framework (e .NET Core/.NET 5+), há uma hierarquia inerente à estrutura das classes que formam o framework. A classe-base para todas as classes em .NET é System.Object, e toda classe em C#, explicitamente herdada de outra classe ou não, deriva de System.Object no final da hierarquia. Por exemplo, se olharmos para algumas classes comuns no .NET, como System.String ou System.Collections.List<T>, ambas têm, no topo de sua hierarquia, a classe System.Object. Isso implica que todos os tipos – sejam valores ou referências – têm um conjunto comum de métodos herdados de System.Object, como ToString(), GetHashCode() e Equals().

O .NET Framework é organizado em namespaces, contêineres lógicos para tipos (classes, enumerações, interfaces, delegados etc.). Cada namespace (que veremos em detalhes no tópico 3.2) é projetado para agrupar funcionalidades semelhantes. Por exemplo:

- **System**: contém tipos fundamentais.
- **System.Collections.Generic**: tipos que suportam coleções fortemente tipadas.
- **System.IO**: tipos que permitem leitura e escrita em fluxos de dados e arquivos.
- **System.Linq**: tipos que suportam consultas usando LINQ.
- **System.Threading**: tipos para multithreading e programação paralela.

A organização em namespaces ajuda a gerenciar e navegar na grande quantidade de classes e tipos no .NET.

Saiba mais

A Microsoft mantém uma documentação detalhada, sempre atualizada, sobre .NET e C# em um de seus sites:

DOCUMENTAÇÃO do C#. *Learn Microsoft*, 26 maio 2017. Disponível em: https://tinyurl.com/yeypc4ww. Acesso em: 23 out. 2023.

2.10 Herança (simples e múltipla) e polimorfismo

Herança é um dos pilares da POO, que permite criar uma nova classe com base em uma classe existente, e essa nova classe herda atributos e comportamentos da classe-base. Em C#, a **herança simples** é codificada utilizando o operador dois-pontos (:). A classe derivada herda membros (métodos, propriedades etc.) da classe-base.

A principal vantagem da herança simples é a capacidade de reutilizar código. Ao criar uma classe-base com características e comportamentos comuns, evita-se a duplicação de código nas classes derivadas.

Em termos de acessibilidade, uma classe derivada tem acesso a todos os membros public (acessíveis por qualquer código) e protected (acessíveis pela classe-base e por qualquer classe derivada) da classe-base. Ela não tem acesso a membros private (acessível apenas pela classe que o declara) da classe-base.

Na hierarquia simples, a relação estabelecida é de "é um". Para entender melhor essa relação, vamos considerar uma representação simplificada da mitologia grega, ilustrada na figura 57. Note que Zeus (linha 10) e Athena (linha 18) derivam da classe Deus. Em suas declarações aparece o símbolo de dois-pontos, sinalizando a herança.

```
1.  public class Deus
2.  {
3.    public string Nome { get; set; }
4.    public void MostrarPoderes()
5.    {
6.      Console.WriteLine("Poderes divinos genéricos.");
7.    }
8.  }
9.
10. public class Zeus : Deus
11. {
12.   public void AtirarRaio()
13.   {
14.     Console.WriteLine("Zeus atira um raio!");
15.   }
16. }
17.
18. public class Athena : Deus
19. {
20.   public void MostrarSabedoria()
21.   {
22.     Console.WriteLine("Athena demonstra sabedoria.");
23.   }
24. }
```

Figura 57 – Classe Deus: exemplo de classe-base e classes derivadas (Zeus e Athena)

O relacionamento entre Zeus e Deus é do tipo "é um", ou seja, Zeus é um deus, assim como Athenas. Ambos herdam o campo Nome e o método MostrarPoderes da classe-base Deus. Além disso, cada um tem características únicas: Zeus tem a habilidade AtirarRaio (linha 12), e Athena, MostrarSabedoria (linha 22). Essas habilidades são independentes entre si.

Overriding é a capacidade de uma classe derivada fornecer implementação específica de um método já definido em sua classe-base. O método na classe derivada deve ter a mesma assinatura que o método na classe-base. Usamos as palavras-chave **virtual** na classe-base para indicar que um método pode ser sobrescrito, e **override** na classe derivada para sobrescrever o método.

A figura 58 ilustra esse método: quando o método Saudacao em um objeto da classe Hades é invocado, a saída será "Eu sou Hades, deus do submundo!", em vez da saudação genérica definida na classe Deus.

```
1.  public class Deus
2.  {
3.     public virtual void Saudacao()
4.     {
5.        Console.WriteLine("Saudação de um deus genérico.");
6.     }
7.  }
8.
9.  public class Hades : Deus
10. {
11.    public override void Saudacao()
12.    {
13.       Console.WriteLine("Eu sou Hades, deus do submundo!");
14.    }
15. }
```

Figura 58 – Classe Deus: exemplo de método sobrescrito na classe derivada Hades

Em C#, uma classe somente pode herdar diretamente de uma única classe, pois isso evita problemas e ambiguidades que podem surgir com a **herança múltipla** que, embora potente, também pode introduzir uma série de complicações e ambiguidades no design orientado a objetos.

A seguir, as razões pelas quais C# optou por não suportar herança múltipla diretamente:

- problema do diamante;
- complexidade de implementação;
- dificuldade do uso;
- quantidade de alternativas disponíveis;
- foco na simplicidade e clareza.

Um dos problemas mais citados com a herança múltipla é o chamado problema do diamante (ou problema do losango). Imagine uma classe-base chamada Deidade. Duas classes, DeusDoCeu e DeusDoMar, herdam de Deidade. Agora suponha uma divindade específica chamada PoseidonZeusHíbrido que, por algum motivo fictício e interessante, herda de ambas as classes, DeusDoCeu e DeusDoMar. Se Deidade tiver um método Saudacao, e tanto DeusDoCeu quanto DeusDoMar o sobrescrevem, o problema do diamante surge: se você chamar Saudacao em um objeto da classe PoseidonZeusHíbrido, qual versão do método ele deve executar? A de DeusDoCeu ou DeusDoMar? Esta é uma simplificação do problema do diamante e é uma das razões pelas quais muitas linguagens, incluindo C#, optam por não suportar herança múltipla diretamente.

A figura 59 ilustra o relacionamento entre essas classes.

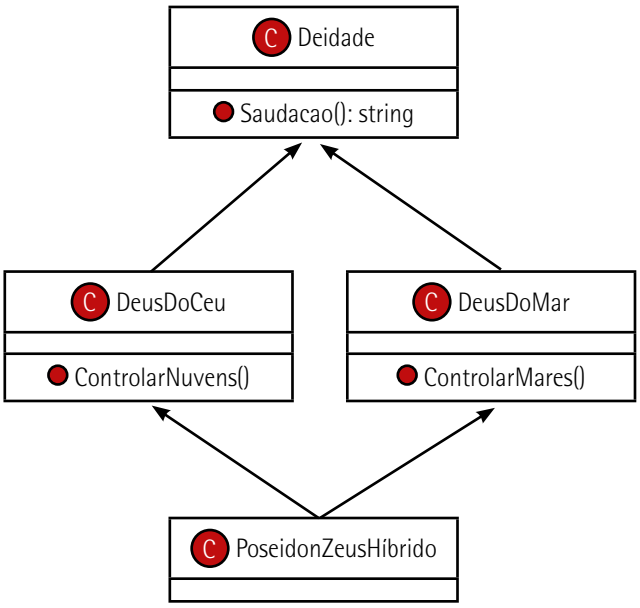

Figura 59 – O problema do diamante: diagrama de classe

Ainda que a herança múltipla seja útil em alguns cenários avançados, pode ser confusa para desenvolvedores menos experientes e levar a erros sutis. Rastrear a origem de um método ou propriedade pode ser muito difícil com a herança múltipla, tornando o código menos legível e mais propenso a erros. O design do C# sempre se concentrou em proporcionar uma linguagem simples, clara e consistente. Evitar a herança múltipla é consistente com essa filosofia, direcionando os desenvolvedores para padrões de design mais simples e claros.

Em vez de herança múltipla, C# oferece uma funcionalidade semelhante, mas mais controlada, através das interfaces (vimos em detalhes no tópico 2.8). Embora as interfaces permitam que uma classe herde múltiplos contratos, elas não carregam implementações, evitando muitos dos problemas associados à herança múltipla. Adicionalmente, com a introdução de métodos de extensão e interfaces com métodos-padrão no C# 8.0, os programadores têm ainda mais flexibilidade sem recorrer à herança múltipla. A figura 60 ilustra como a classe Hermes implementa as interfaces IVoador e IGuerreiro, dispensando a necessidade de herança múltipla.

```
1.  public interface IVoador
2.  {
3.    void Voar();
4.  }
5.  public interface IGuerreiro
6.  {
7.    void Lutar();
8.  }
9.  public class Hermes : IVoador, IGuerreiro
10. {
11.   public void Voar()
12.   {
13.     Console.WriteLine("Hermes voa rapidamente.");
14.   }
15.
16.   public void Lutar()
17.   {
18.     Console.WriteLine("Hermes luta com agilidade.");
19.   }
20. }
```

Figura 60 – Interface IVoador: exemplo de estratégia sem herança múltipla

Em resumo, mesmo que a herança múltipla tenha seus usos e seja suportada em algumas linguagens, a equipe de design do C# optou por não a suportar diretamente, evitando os problemas associados e incentivando padrões de design mais claros e sustentáveis. Herança simples é uma ferramenta poderosa no arsenal de um desenvolvedor que permite uma modelagem clara e uma reutilização eficiente do código. Ao compreender profundamente como a hierarquia de classes funciona em C#, desenvolvedores podem criar sistemas mais organizados, escaláveis e manuteníveis.

A palavra-chave base em C# é frequentemente usada em contextos de herança para se referir a membros da classe-base a partir da classe derivada. É especialmente útil quando se deseja acessar construtores, métodos, propriedades ou campos da classe-base que foram ocultados ou sobrescritos na classe derivada.

```
1.  public class Deus
2.  {
3.    public string Nome { get; private set; }
4.
5.    public Deus(string nome)
6.    {
7.      this.Nome = nome;
8.    }
9.
10.   public virtual void Apresentar()
11.   {
12.     Console.WriteLine($"Eu sou {Nome}, um deus do Olimpo.");
13.   }
14. }
15.
16. public class Zeus : Deus
17. {
18.   public string Arma { get; private set; }
19.
20.   public Zeus(string nome, string arma) : base(nome)  // Uso da
palavra-chave 'base' para invocar o construtor da classe Deus.
21.   {
22.     this.Arma = arma;
23.   }
24.
25.   public override void Apresentar()
26.   {
27.     base.Apresentar();  // Chama o método Apresentar() da classe base
Deus.
28.     Console.WriteLine($"Eu empunho o {Arma}!"); // Adiciona
informação extra sobre Zeus.
29.   }
30. }
31.
32. public static void Main(string[] args)
33. {
34.   Zeus zeus = new Zeus("Zeus", "trovão");
35.   zeus.Apresentar();
36. }
```

Figura 61 – Classe Deus: exemplo da utilização de base

Temos uma classe-base Deus com um construtor que define o nome do deus e um método Apresentar. A classe Zeus herda de Deus e quer não apenas utilizar a funcionalidade do construtor da classe-base, mas também adicionar sua própria funcionalidade ao método Apresentar. No exemplo, se criarmos uma instância de Zeus e invocarmos o método Apresentar, a saída será: "Eu sou Zeus, um deus do Olimpo.", e na linha seguinte do console "Eu empunho o trovão!". No construtor de Zeus, usamos base(nome) para chamar o construtor da classe Deus. Além disso, dentro do método Apresentar de Zeus, chamamos base. Apresentar para executar a implementação original de Apresentar da classe Deus antes de adicionar nossa própria lógica extra para Zeus.

Polimorfismo é um dos quatro pilares da POO (ao lado de encapsulamento, herança e abstração). Em sua essência, permite tratar objetos derivados como instâncias de sua classe-base, possibilitando a escrita de um código mais genérico e reutilizável. Em C#, o polimorfismo é expresso principalmente de duas maneiras: através da sobrecarga (overloading) de métodos e da substituição (overriding) de métodos. O polimorfismo de substituição (overriding) é uma forma de polimorfismo de tempo de execução e se relaciona à herança (vide exemplo na figura 58).

```
1.  using System;
2.  // Classe base
3.  public class Deidade
4.  {
5.     // Método virtual que permite sua substituição nas classes
derivadas
6.     public virtual void ExibirHabilidade()
7.     {
8.        Console.WriteLine("Eu sou uma divindade com poderes
misteriosos.");
9.  }
10. }
11. // Classe derivada
12. public class Zeus : Deidade
13. {
14.    // Sobrescrevendo o método da classe base
15.    public override void ExibirHabilidade()
16.    {
17.       Console.WriteLine("Eu sou Zeus, e posso lançar raios!");
18.    }
19. }
20. // Outra classe derivada
21. public class Poseidon : Deidade
22. {
23.    // Sobrescrevendo o método da classe base
24.    public override void ExibirHabilidade()
25.    {
26.       Console.WriteLine("Eu sou Poseidon, e posso controlar os
mares!");
27.    }
28. }
29. // Classe derivada
30. public class Athena : Deidade
31. {
32.    // Sobrescrevendo o método da classe base
33.    public override void ExibirHabilidade()
34.    {
35.       Console.WriteLine("Eu sou Athena, e ofereço sabedoria e
estratégia em batalha!");
36.    }
37. }
```

Figura 62 – Classe Deidade: exemplo de polimorfismo de substituição do método ExibirHabilidade em três classes derivadas (Zeus, Poseidon e Athena)

Temos uma classe-base Deidade com um método ExibirHabilidade. Esse método é sobrescrito em várias classes derivadas (Zeus, Poseidon, Athena) para refletir suas habilidades únicas.

```
1.  public class Program
2.  {
3.    public static void Main()
4.    {
5.      Deidade zeus = new Zeus();
6.      Deidade poseidon = new Poseidon();
7.      Deidade athena = new Athena();
8.
9.      zeus.ExibirHabilidade();   // Saída: Eu sou Zeus, e posso lançar
raios!
10.     poseidon.ExibirHabilidade(); // Saída: Eu sou Poseidon, e posso
controlar os mares!
11.     athena.ExibirHabilidade();  // Saída: Eu sou Athena, e ofereço
sabedoria e estratégia em batalha!
12.   }
13. }
```

Figura 63 – Classe Deidade: invocação do método ExibirHabilidade em três objetos

Quando invocamos o método ExibirHabilidade (figura 63) em um objeto de uma dessas classes derivadas, o método sobrescrito é chamado, demonstrando o polimorfismo de substituição (overriding).

```
1.  using System;
2.  public class Deus
3.  {
4.    public string Nome { get; private set; }
5.
6.    public Deus(string nome)
7.    {
8.      Nome = nome;
9.    }
10.   // Método para receber uma oferenda sem especificação
11.   public void ReceberOferenda()
12.   {
13.     Console.WriteLine($"{Nome} agradece sua oferenda!");
14.   }
15.
16.   // Sobrecarga para receber uma oferenda especificando o tipo
17.   public void ReceberOferenda(string item)
18.   {
19.     Console.WriteLine($"{Nome} agradece sua oferenda de {item}!");
20.   }
21.   // Sobrecarga para receber uma oferenda especificando o tipo e
quantidade
22.   public void ReceberOferenda(string item, int quantidade)
23.   {
24.     Console.WriteLine($"{Nome} agradece sua oferenda de {quantidade}
{item}(s)!");
25.   }
26.
27.   // Sobrecarga para receber uma oferenda especificando o valor em
ouro
28.   public void ReceberOferenda(double valorEmOuro)
29.   {
30.     Console.WriteLine($"{Nome} agradece sua generosa oferenda de
{valorEmOuro} moedas de ouro!");
31.   }
32. }
```

Figura 64 – Classe Deus: exemplo de polimorfismo de sobrecarga (overloading) do método ReceberOferenda

Quando uma classe derivada fornece uma nova implementação para um método que já existe em sua classe-base, dizemos que ela está sobrescrevendo esse método. Isso permite que um objeto de uma classe derivada seja tratado como objeto de sua classe-base, mas ainda assim invoque o método apropriado da classe derivada.

A figura 64 ilustra o polimorfismo de sobrecarga (overloading), uma forma de polimorfismo de tempo de compilação que se refere à capacidade de uma única classe ter múltiplos métodos (ou construtores) com o mesmo nome, mas com diferentes listas de parâmetros (vimos um exemplo na figura 36). O compilador determina qual versão do método deve ser chamada com base nos argumentos fornecidos.

Na figura 64, temos a classe Deus com diferentes sobrecargas do método ReceberOferenda. Como podemos observar nas linhas 7, 8, 9 e 10 da figura 65, dependendo de como chamamos esse método (com diferentes argumentos), uma versão diferente dele é executada; isso é overloading (sobrecarga) em ação. A decisão sobre qual versão do método executar é tomada em tempo de compilação com base nos argumentos fornecidos.

```
1.  public class Program
2.  {
3.    public static void Main()
4.    {
5.      Deus zeus = new Deus("Zeus");
6.
7.      zeus.ReceberOferenda();   // Saída: Zeus agradece sua oferenda!
8.      zeus.ReceberOferenda("vinho"); // Saída: Zeus agradece sua
oferenda de vinho!
9.      zeus.ReceberOferenda("vinho", 3);  // Saída: Zeus agradece sua
oferenda de 3 vinho(s)!
10.     zeus.ReceberOferenda(100.5); // Saída: Zeus agradece sua generosa
oferenda de 100.5 moedas de ouro!
11.   }
12. }
```

Figura 65 – Classe Deus: invocação do método
ReceberOferenda de quatro passagens de parâmetro distintas

Resumo

Nesta unidade apresentamos as bases da programação orientada a objetos (POO). Iniciamos pela fundamentação e motivação, passando pelo desenvolvimento dos conceitos de classe e objeto, fundamentais para o processo. Entender claramente como uma classe é modelada e como um objeto é concebido, bem como as diferenças entre os dois, é de extrema importância prática.

Também vimos os quatro pilares da POO: abstração, encapsulamento, herança e polimorfismo. Por serem pilares, tanto as noções elementares quanto os exemplos de implantação são importantes para fixar seus conceitos, distintos da tradicional programação estruturada. Além da apresentação geral, descrevemos com riqueza de exemplos de código a criação de classes, objetos, atributos, campos e constantes, apontando a importância dos métodos e suas variações; em especial, descrevemos a utilização da referência this, as classes estáticas, interfaces e as classes abstratas.

Por fim, tratamos da hierarquia de classes e herança simples e múltipla – esta não será usada no C#, mas é necessário conhecê-la e saber os motivos de sua não adoção pela linguagem. Todos esses conceitos são importantes para as próximas unidades, portanto precisam estar devidamente consolidados para uma boa absorção do que está por vir.

Exercícios

Questão 1. (Cespe-Cebraspe 2019, adaptada) Considerando os modelos de programação estruturada (PE) e de POO, avalie as afirmativas.

I – Sob o paradigma de PE, um código de programa consiste em sub-rotinas (funções e procedimentos) associadas a uma rotina principal.

II – Sob o paradigma de POO, um código de programa consiste em diversos objetos inter-relacionados por métodos que estabelecem um nível de comunicação (colaboração entre objetos).

III – Os pilares da orientação a objetos são classe, objeto, atributo e método.

IV – Na POO, uma característica das interfaces é o fato de não poderem ser implementadas por uma classe, mas sim herdadas.

É correto o que se afirma em:

A) I, apenas.

B) III, apenas.

C) I e II, apenas.

D) II e IV, apenas.

E) I, II, III e IV.

Resposta correta: alternativa C.

Análise das afirmativas

I – Afirmativa correta.

Justificativa: no paradigma estruturado, que utilizamos quando programamos em linguagem C, um programa é geralmente dividido em funções e procedimentos, que são chamados a partir de uma função principal.

II – Afirmativa correta.

Justificativa: no paradigma orientado a objeto, que utilizamos quando trabalhamos com linguagem C#, o programa consiste em objetos que se comunicam por métodos. Os objetos são instanciados a partir de classes.

III – Afirmativa incorreta.

Justificativa: os quatro pilares da POO são: abstração, encapsulamento, herança e polimorfismo. Com eles os desenvolvedores são capazes de criar programas cuja estrutura se assemelha à de objetos do mundo real, resultando em softwares intuitivos. Os conceitos de classe, objeto, atributo e método são importantes na POO, mas não são considerados pilares.

IV – Afirmativa incorreta.

Justificativa: em POO, uma interface determina um conjunto de métodos que uma classe deve implementar. Consequentemente, as interfaces são implementadas (não herdadas) por uma classe. Classes que implementam uma interface devem fornecer a implementação para todos os métodos declarados na interface.

Questão 2. (Instituto Consulplan 2023, adaptada) Na POO, herança é uma forma de reutilizar software em que uma nova classe é criada absorvendo membros de uma classe existente. Essa nova classe é aprimorada com capacidades novas ou modificadas.

Em relação à herança, assinale a alternativa correta:

A) A classe existente é chamada de subclasse, e a nova classe é chamada de superclasse.

B) Uma das suas desvantagens é que uma subclasse não pode adicionar seus próprios campos e métodos.

C) Com a herança, o programador pode economizar tempo enquanto desenvolve um programa.

D) Para evitar inconsistências durante o desenvolvimento de sistemas, jamais o objeto de uma classe pode ser objeto de outra classe.

E) As relações de herança não formam estruturas hierárquicas, uma vez que são classificadas como estruturas puramente lineares, ou seja, todas as classes estão sempre no mesmo nível.

Resposta correta: alternativa C.

Análise das alternativas

A) Alternativa incorreta.

Justificativa: a classe já existente é chamada de superclasse, enquanto a nova classe é denominada subclasse.

B) Alternativa incorreta.

Justificativa: uma das vantagens da herança é que uma subclasse pode ter seus próprios campos e métodos, além de herdar os campos e os métodos da classe que a originou.

C) Alternativa correta.

Justificativa: a herança permite que a nova classe (subclasse) absorva membros e comportamentos da classe já existente (superclasse). Isso facilita a reutilização de código, o que leva à economia do tempo de desenvolvimento.

D) Alternativa incorreta.

Justificativa: na POO é possível que o objeto de uma classe seja usado em objetos de outras classes, desde que isso seja feito de maneira apropriada.

E) Alternativa incorreta.

Justificativa: as relações de herança formam, sim, estruturas hierárquicas, em que as classes podem estar em diferentes níveis de herança. Uma classe pode herdar de outra, criando uma relação de hierarquia entre elas. Essa hierarquia pode inclusive ser estendida para vários níveis.